

AIE683 Machine Learning

Week 4: Linear Models: Regression, Regularization & Classification

Dr. Hakan Emekci

Hacettepe University, Graduate School of Informatics

Two prediction problems

What is this house worth?

- Automated valuation models price homes for lenders, insurers and tax assessors
- Errors are measured in money, and they are asymmetric in consequence
- Ames, Iowa: 2,930 sales, 80 features

Is this tumour malignant?

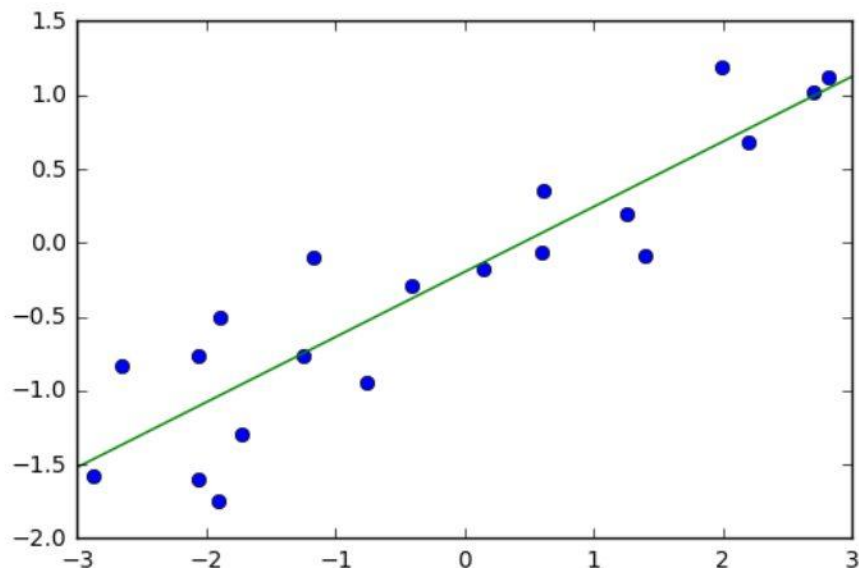
- Diagnosis from 30 measurements of a biopsy image
- The label is not enough: a doctor needs a calibrated probability
- Wisconsin breast cancer data: 569 cases

Plan for today

1. Linear regression, Ridge, Lasso, Elastic Net
2. Try it 1: a small Ames competition
3. Linear classification: logistic regression and linear SVMs, multiclass
4. Kernel SVMs
5. Try it 2: predict, then run
6. Tracking regularization paths and model comparisons in MLflow

Linear Models for Regression

Linear Models for Regression



$$\hat{y} = \mathbf{w}^T \mathbf{x} + b = \sum_{i=1}^p w_i x_i + b$$

Ordinary Least Squares

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b = \sum_{i=1}^p w_i x_i + b$$

$$\min_{\mathbf{w} \in \mathbb{R}^p, b \in \mathbb{R}} \sum_{i=1}^n (\mathbf{w}^T \mathbf{x}_i + b - y_i)^2$$

Unique solution if $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_n)^T$ has full column rank.

Ridge Regression

$$\min_{\mathbf{w} \in \mathbb{R}^p, b \in \mathbb{R}} \sum_{i=1}^n (\mathbf{w}^T \mathbf{x}_i + b - y_i)^2 + \alpha \|\mathbf{w}\|^2$$

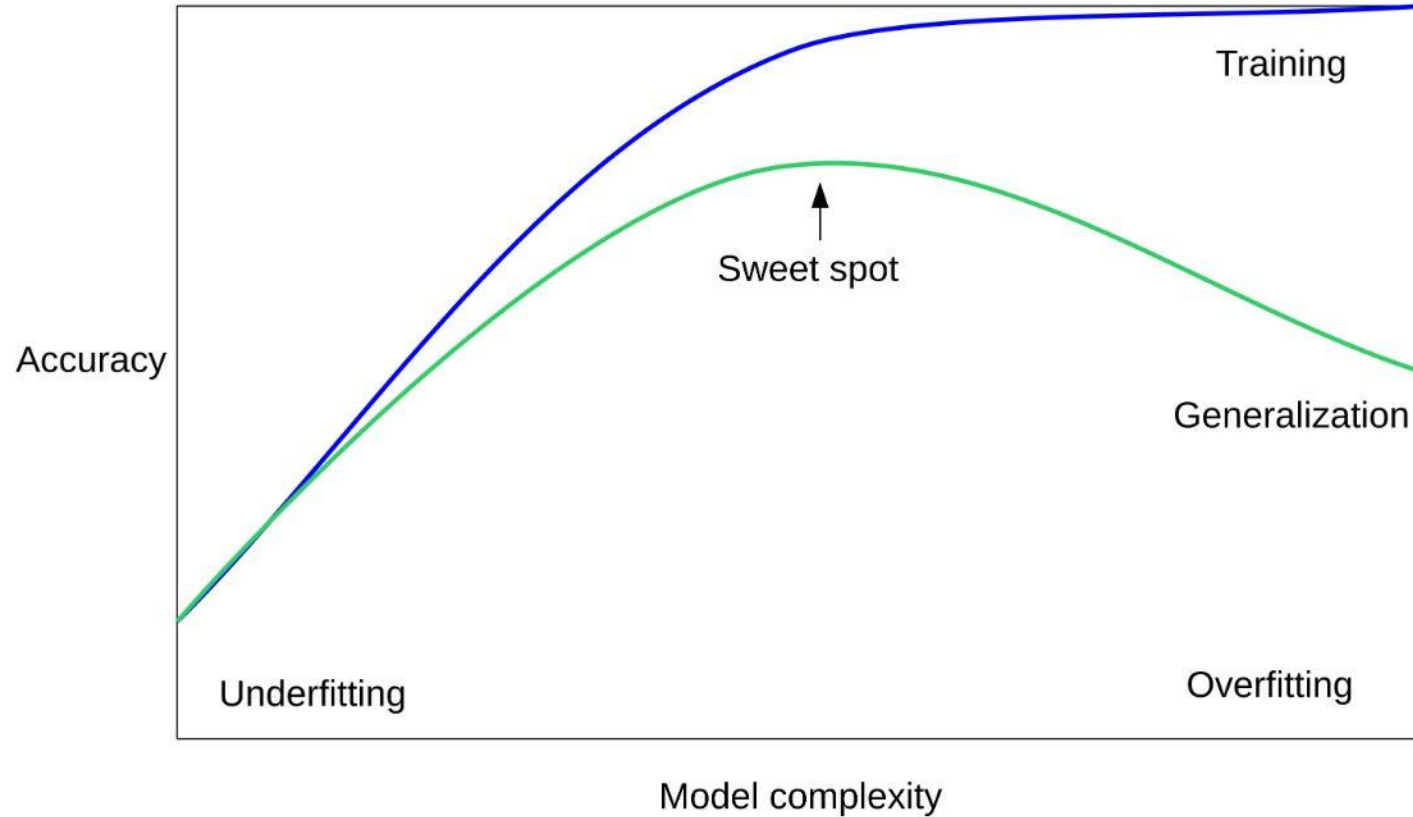
Always has a unique solution.

Tuning parameter alpha.

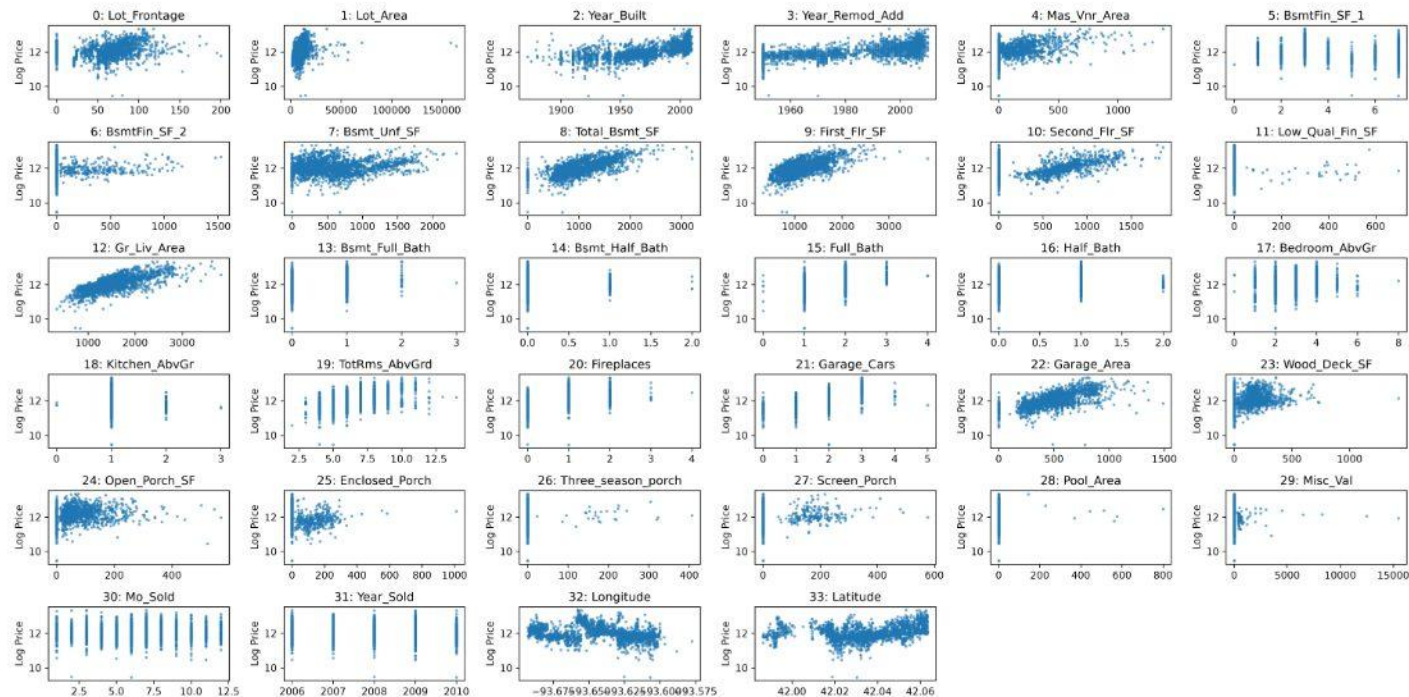
(regularized) Empirical Risk Minimization

$$\min_{f \in \mathcal{F}} \underbrace{\sum_{i=1}^n L(f(\mathbf{x}_i), y_i)}_{\text{data fitting}} + \underbrace{\alpha R(f)}_{\text{regularization}}$$

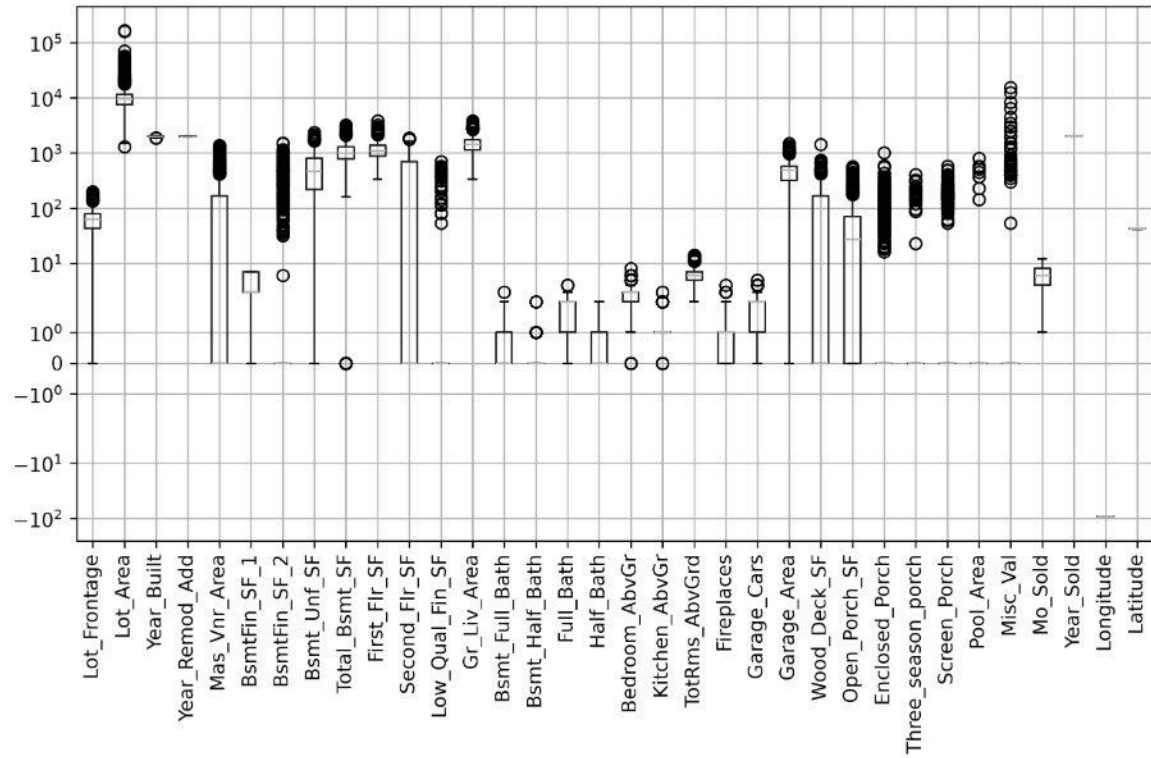
Reminder on model complexity



Ames Housing Dataset



```
ames = fetch_openml(data_id=43926, as_frame=True) # "ames_housing"
X, y = ames.data, ames.target
X, y = X[X.Gr_Liv_Area <= 4000], y[X.Gr_Liv_Area <= 4000]
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=2)
print(X_train.shape) # (2193, 80)
```



Coefficient of determination R^2

$$R^2(y, \hat{y}) = 1 - \frac{\sum_{i=0}^{n-1} (y_i - \hat{y}_i)^2}{\sum_{i=0}^{n-1} (y_i - \bar{y})^2}$$

$$\bar{y} = \frac{1}{n} \sum_{i=0}^{n-1} y_i$$

Can be negative for biased estimators - or the test set!

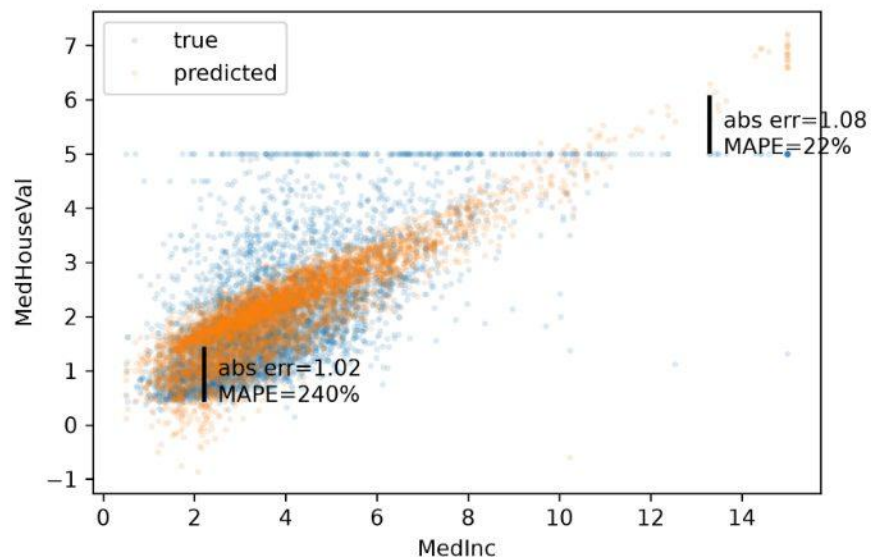
Metrics for regression models

Built-in standard metrics

- R^2 : easy to understand scale
- MSE / RMSE : same units as the target (RMSE)
- Mean absolute error, median absolute error: more robust
- MAPE: relative error (`mean_absolute_percentage_error`)
- D^2 scores: R^2 -style skill score for other losses (`d2_absolute_error_score`, ...)

Absolute vs Relative: MAPE

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{y - \hat{y}}{y} \right|$$



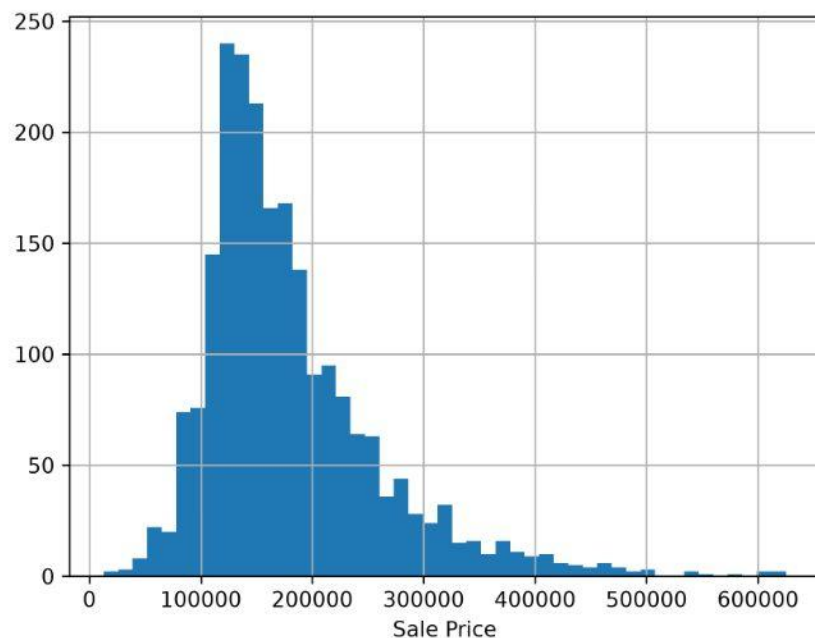
Preprocessing

```
cat_preprocessing = make_pipeline(  
    SimpleImputer(strategy='constant', fill_value='NA'),  
    OneHotEncoder(handle_unknown='ignore'))  
  
cont_preprocessing = make_pipeline(  
    SimpleImputer(),  
    StandardScaler())  
  
preprocess = make_column_transformer(  
    (cat_preprocessing, make_column_selector(dtype_include=['object', 'category']))  
    remainder=cont_preprocessing)  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, random_state=2)  
cross_val_score(  
    make_pipeline(preprocess, LinearRegression()),  
    X_train, y_train, cv=5)
```

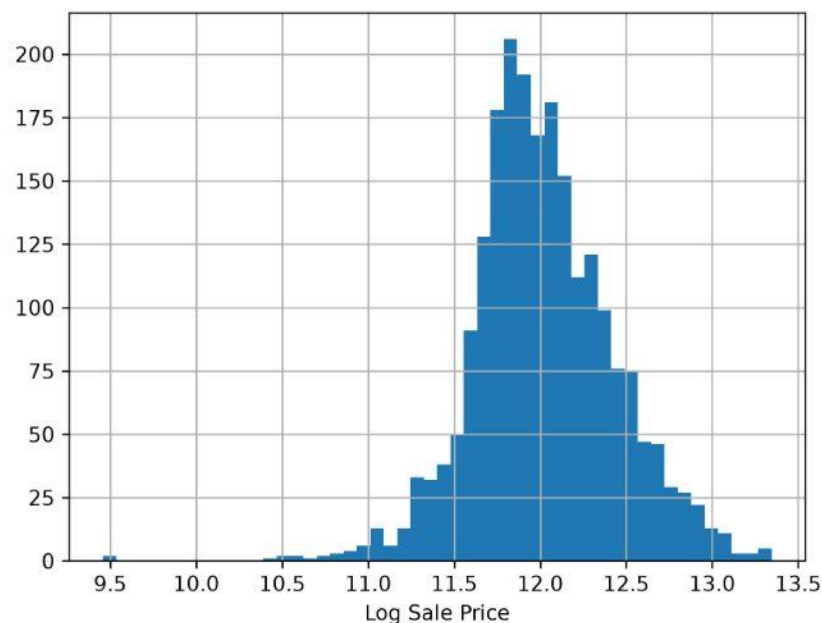
```
array([0.934, 0.926, 0.931, 0.891, 0.897])
```

Skewed targets

```
y_train.hist(bins='auto')
```



```
np.log(y_train).hist(bins='auto')
```



Log target and Ridge

```
from sklearn.compose import TransformedTargetRegressor
log_regressor = TransformedTargetRegressor(
    LinearRegression(), func=np.log, inverse_func=np.exp)
cross_val_score(make_pipeline(preprocess, log_regressor), X_train, y_train, cv=5)
```

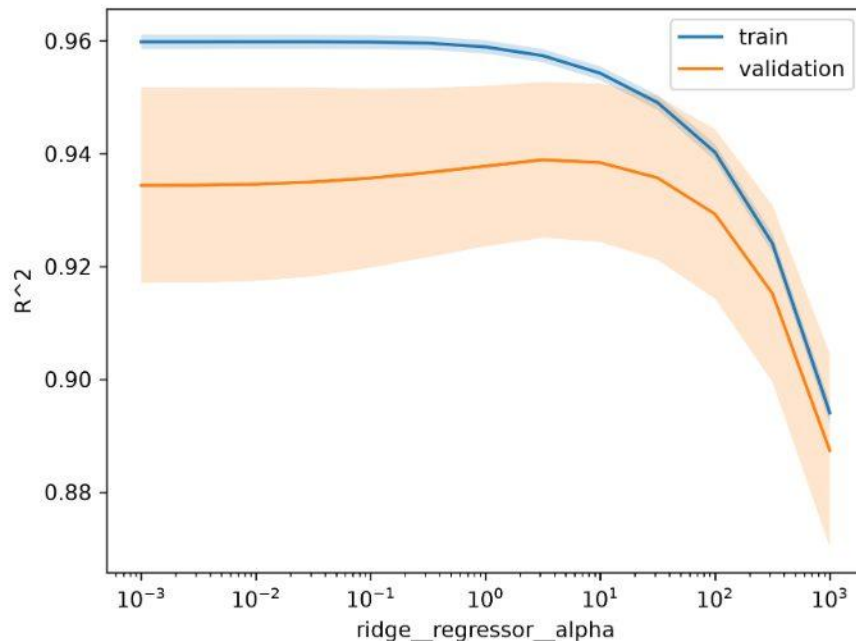
```
array([0.95 , 0.937, 0.936, 0.911, 0.917])
```

```
log_ridge = TransformedTargetRegressor(
    Ridge(), func=np.log, inverse_func=np.exp)
cross_val_score(make_pipeline(preprocess, log_ridge), X_train, y_train, cv=5)
```

```
array([0.948, 0.949, 0.94 , 0.918, 0.931])
```

```
pipe = Pipeline([('preprocessing', preprocess), ('ridge', log_ridge)])  
param_grid = {'ridge__regressor__alpha': np.logspace(-3, 3, 13)}  
  
grid = GridSearchCV(pipe, param_grid, cv=RepeatedKFold(n_splits=10, n_repeats=5),  
                    return_train_score=True)  
grid.fit(X_train, y_train)  
print(grid.best_params_, grid.best_score_)
```

{'ridge__regressor__alpha': 3.162} 0.939



Try it 1: a small Ames competition (15 min)

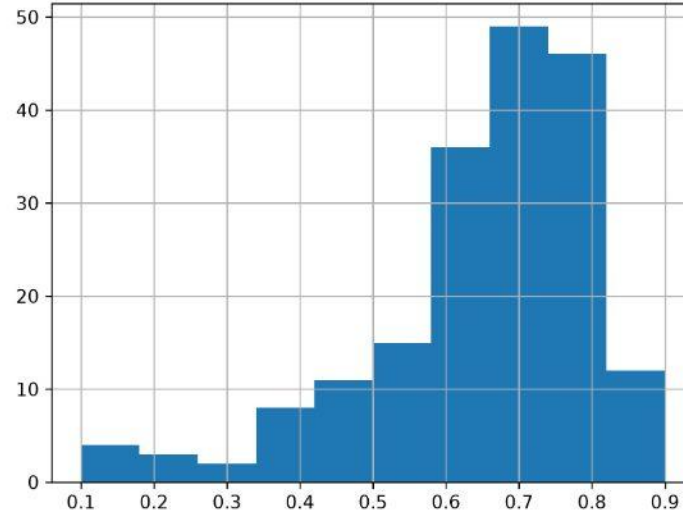
- Goal: lowest **test RMSE in dollars** on the held-out Ames test set
- Linear models only: `Ridge`, `Lasso`, `ElasticNet`, their `...CV` versions
- Any preprocessing: `log target`, `SplineTransformer` or `PolynomialFeatures` on a few columns, adding or dropping features
- Choose everything with cross-validation on the training set; touch the test set **once**
- Lab: section "Try it 1" in `lab.ipynb` (baseline: CV RMSE about \$19,600)

Triazine Dataset

```
triazines = fetch_openml('triazines', version=1, as_frame=True)  
triazines.data.shape
```

(186, 60)

```
triazines.target.hist()
```



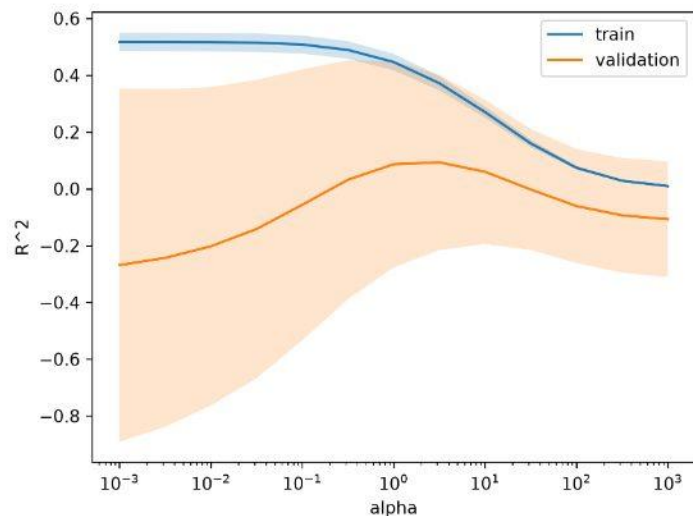
```
X_train, X_test, y_train, y_test = train_test_split(
    triazines.data, triazines.target, random_state=0)
cross_val_score(LinearRegression(), X_train, y_train, cv=5)
```

```
array([ 0.213,  0.129, -0.796, -0.222, -0.155])
```

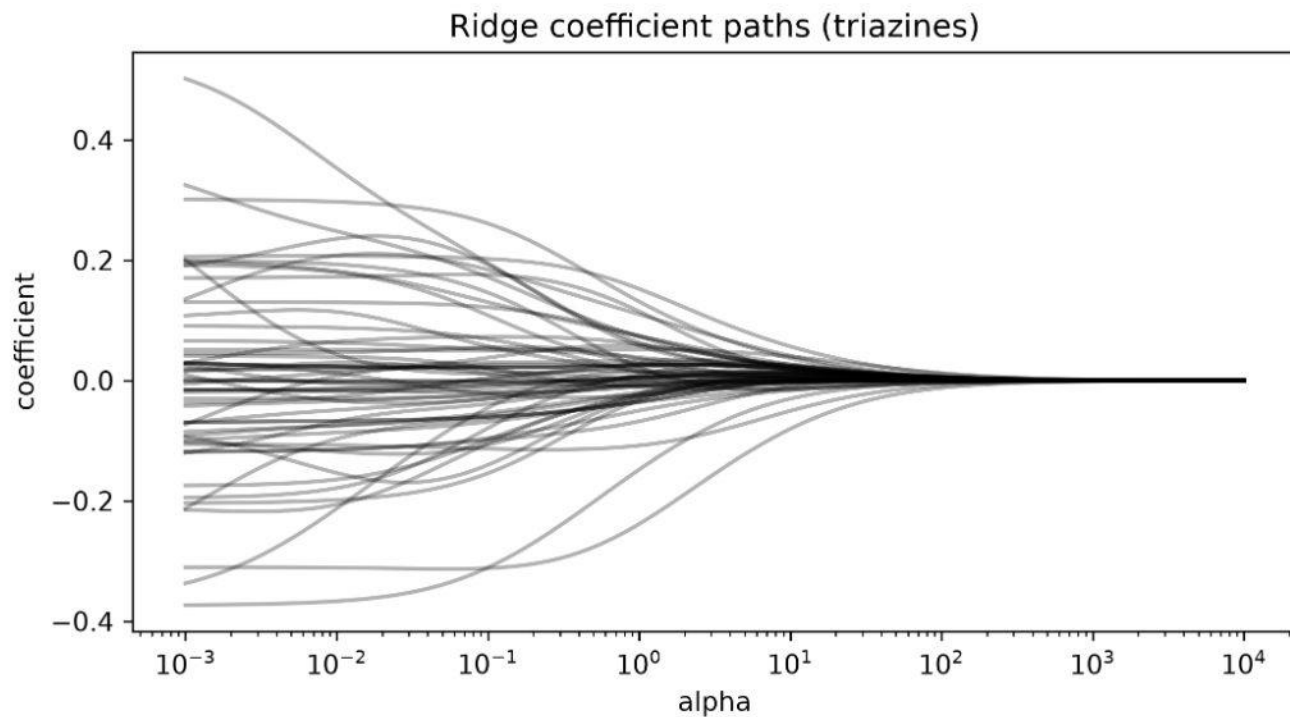
```
cross_val_score(Ridge(), X_train, y_train, cv=5)
```

```
array([0.263, 0.455, 0.024, 0.23 , 0.036])
```

```
param_grid = {'alpha': np.logspace(-3, 3, 13)}  
  
grid = GridSearchCV(Ridge(), param_grid, cv=RepeatedKFold(n_splits=10,  
                                                           return_train_score=True))  
grid.fit(X_train, y_train)
```



Coefficient paths

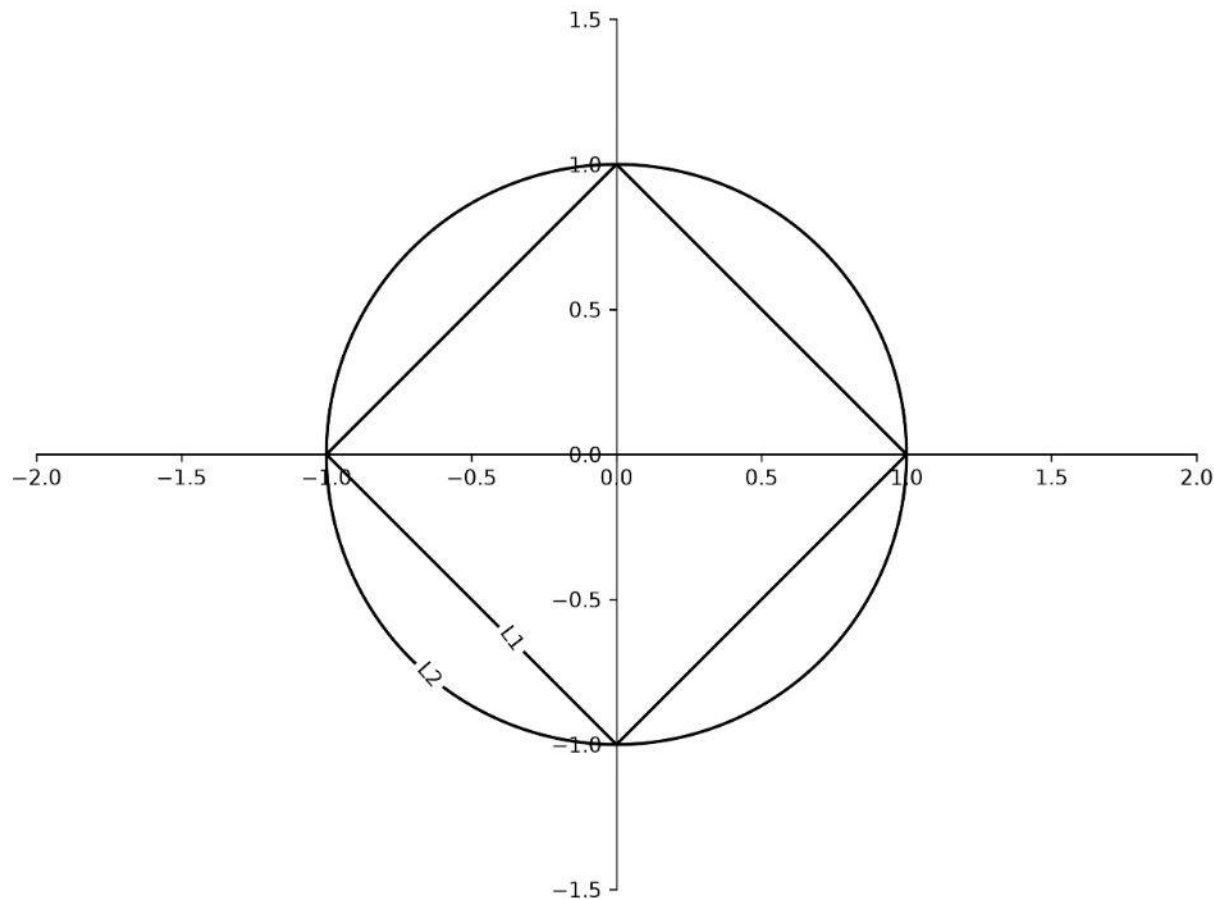


Lasso Regression

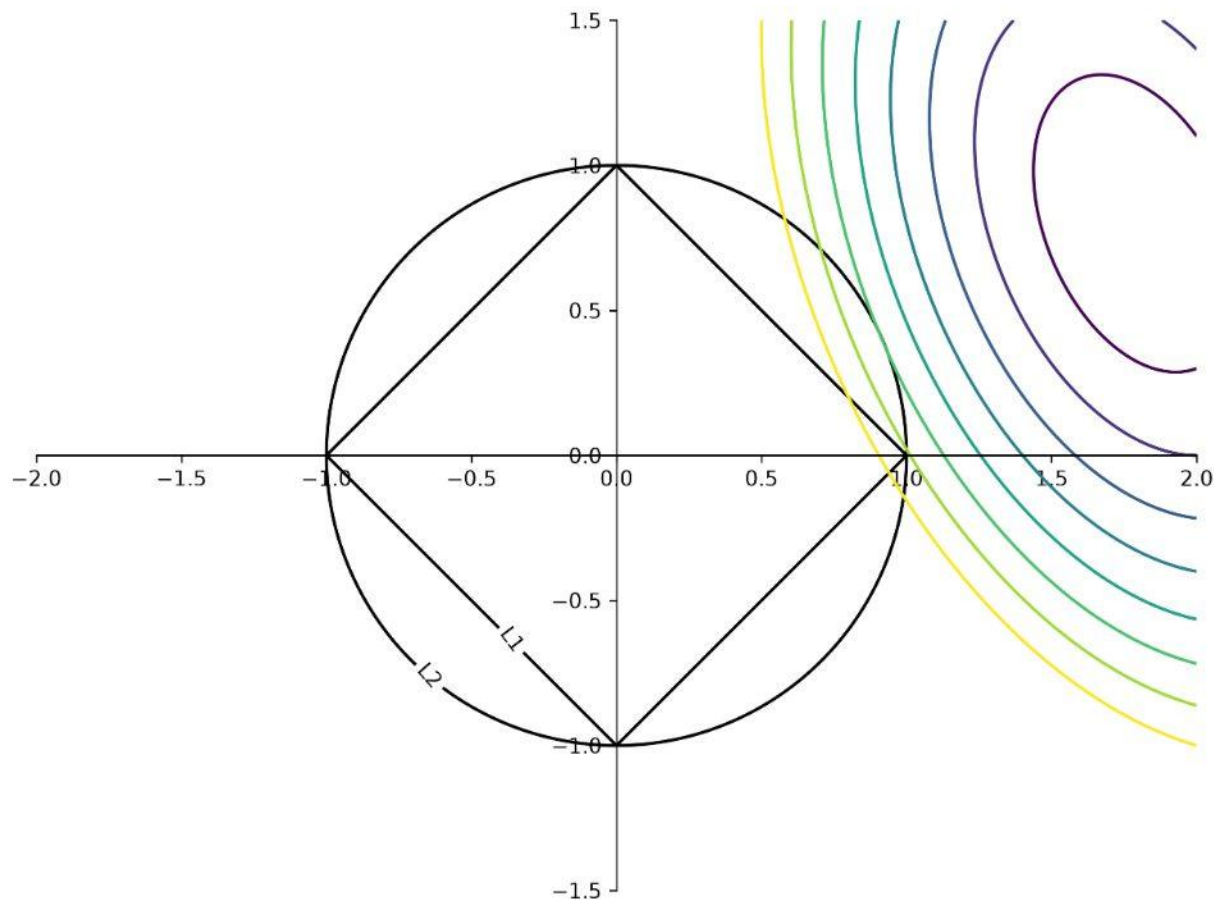
$$\min_{w \in \mathbb{R}^p, b \in \mathbb{R}} \sum_{i=1}^n (w^T \mathbf{x}_i + b - y_i)^2 + \alpha ||w||_1$$

- Shrinks w towards zero like Ridge
- Sets some w exactly to zero - automatic feature selection!

Understanding L1 and L2 Penalties



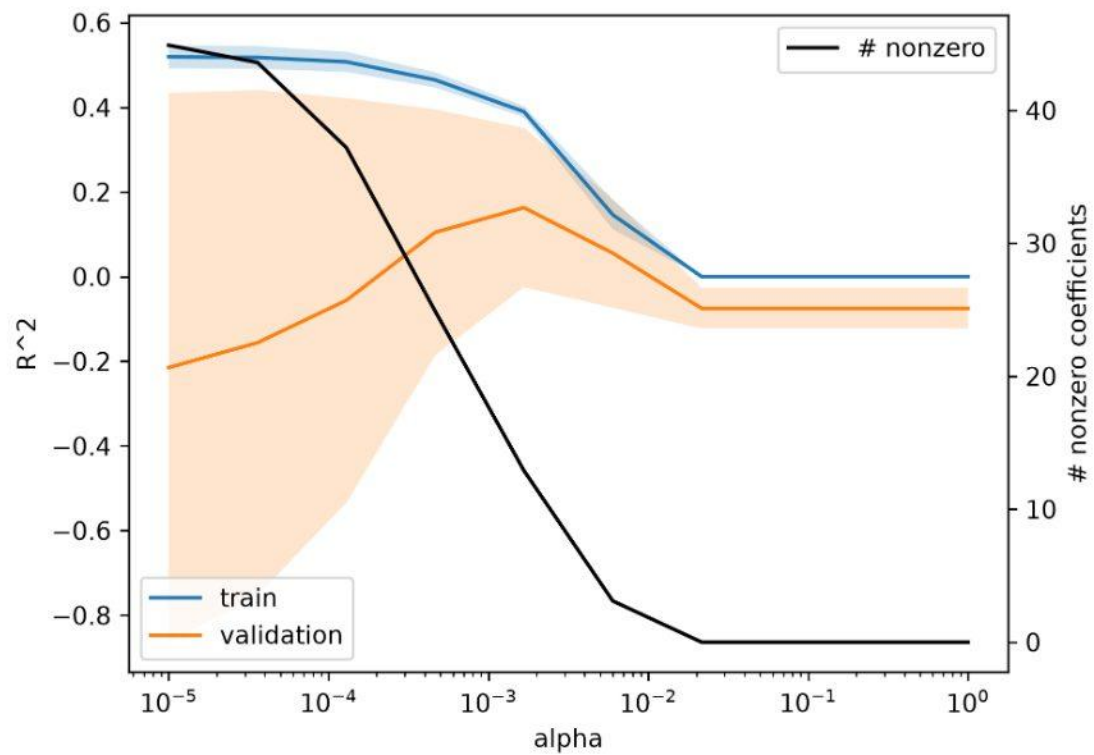
Understanding L1 and L2 Penalties

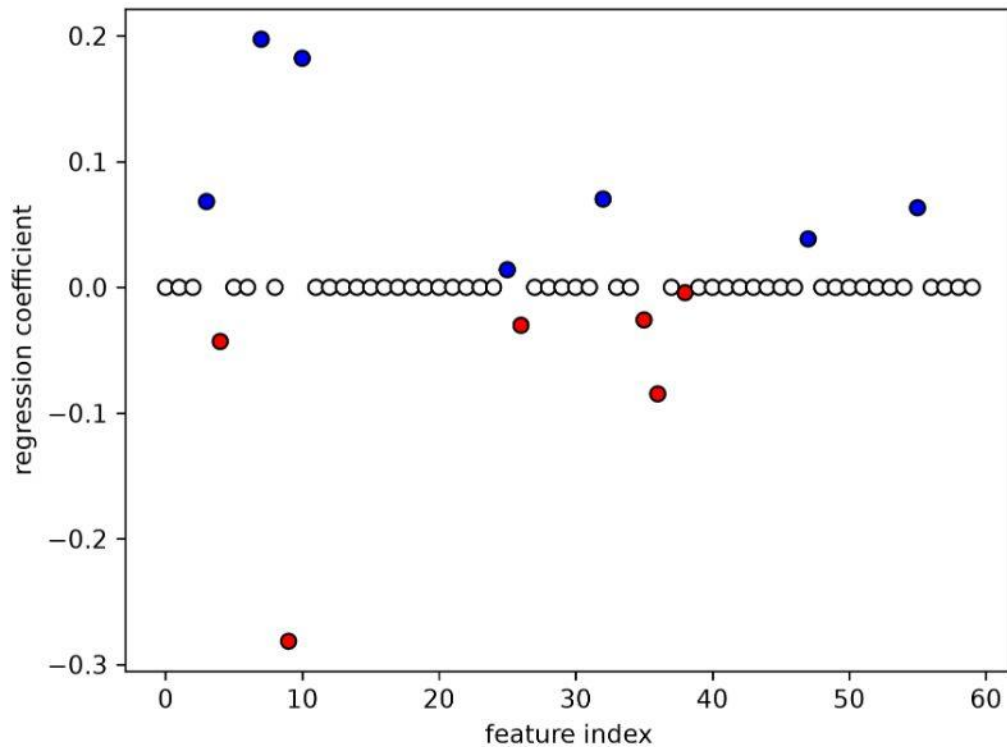


Grid-Search for Lasso

```
param_grid = {'alpha': np.logspace(-5, 0, 10)}  
grid = GridSearchCV(Lasso(max_iter=100_000), param_grid, cv=10)  
grid.fit(X_train, y_train)  
  
print(grid.best_params_)  
print(grid.best_score_)
```

```
{'alpha': 0.0017}  
0.163
```





```
print(X_train.shape)  
np.sum(lasso.coef_ != 0)
```

```
(139, 60)  
13
```

What goes wrong in practice

- **Unstable selection:** of two correlated features, Lasso keeps one, somewhat arbitrarily. Refit on a bootstrap sample and the selected set changes
- **"Zero" is not "irrelevant":** a dropped feature may be as predictive as a kept one
- **Signs flip with alpha:** regularized coefficients are not causal effects
- **Forgotten scaling:** without standardization, the penalty depends on the units of each feature
- **alpha is not comparable** between Ridge, Lasso and ElasticNet (different scaling of the loss)

Elastic Net

- Combines benefits of Ridge and Lasso
- two parameters to tune.

$$\min_{\mathbf{w} \in \mathbb{R}^p, b \in \mathbb{R}} \sum_{i=1}^n (\mathbf{w}^T \mathbf{x}_i + b - y_i)^2 + \alpha_1 ||\mathbf{w}||_1 + \alpha_2 ||\mathbf{w}||_2^2$$

Grid-searching ElasticNet

```
from sklearn.linear_model import ElasticNet
param_grid = {'alpha': np.logspace(-4, -1, 10),
              'l1_ratio': [0.01, .1, .5, .8, .9, .95, .98, 1]}

grid = GridSearchCV(ElasticNet(max_iter=100_000), param_grid, cv=10)
grid.fit(X_train, y_train)

print(grid.best_params_)
print(grid.best_score_)
```

```
{'alpha': 0.0022, 'l1_ratio': 0.5}
0.174
```

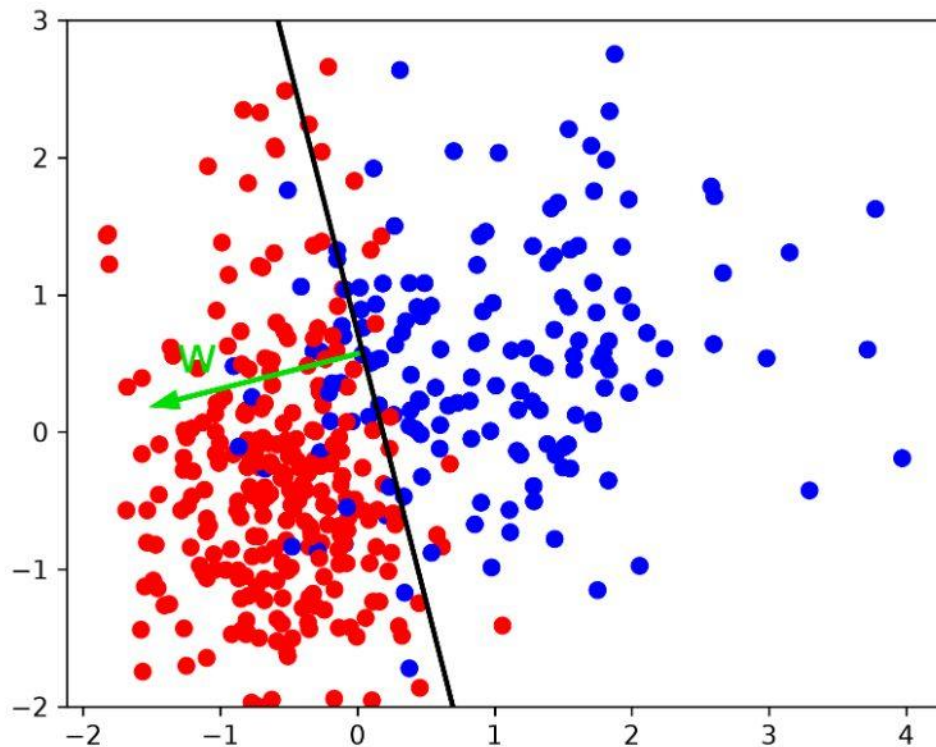
```
(grid.best_estimator_.coef_ != 0).sum()
```

Tracking the regularization path in MLflow

```
mlflow.set_experiment("week04-linear-models")
with mlflow.start_run(run_name="regularization-path-triazines"):
    for name, make_model in [("ridge", lambda a: Ridge(alpha=a)),
                             ("lasso", lambda a: Lasso(alpha=a, max_iter=100_000))]:
        for step, a in enumerate(np.logspace(-4, 2, 13)):
            scores = cross_val_score(make_model(a), X_train, y_train, cv=cv)
            with mlflow.start_run(run_name=f"{name}-alpha={a:.4g}", nested=True):
                mlflow.log_params({"model": name, "alpha": a})
                mlflow.log_metrics({"cv_r2_mean": scores.mean(), "cv_r2_std": scores.std()})
                mlflow.log_metric(f"{name}_cv_r2", scores.mean(), step=step)
            mlflow.log_figure(fig, "regularization_path.png")
```

Part 2: Linear Models for Classification, SVMs

Linear models for binary classification

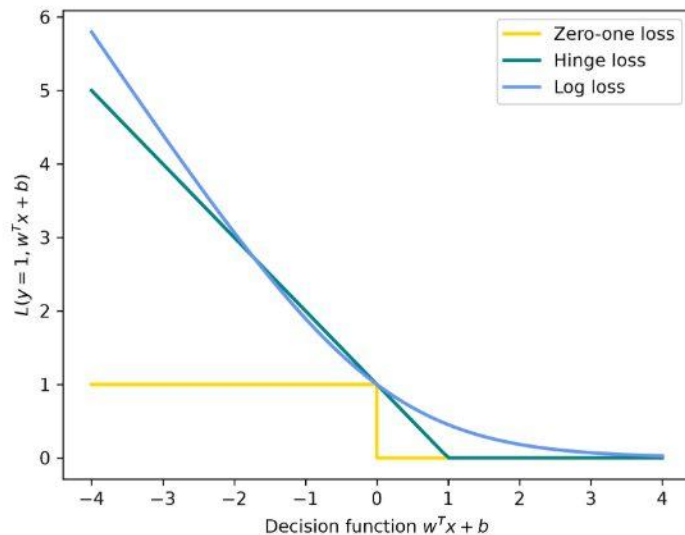


$$\hat{y} = \text{sign}(w^T \mathbf{x} + b) = \text{sign} \left(\sum_i w_i x_i + b \right)$$

Picking a loss?

$$\hat{y} = \text{sign}(w^T \mathbf{x} + b)$$

$$\min_{w \in \mathbb{R}^p, b \in \mathbb{R}} \sum_{i=1}^n 1_{y_i \neq \text{sign}(w^T \mathbf{x}_i + b)}$$



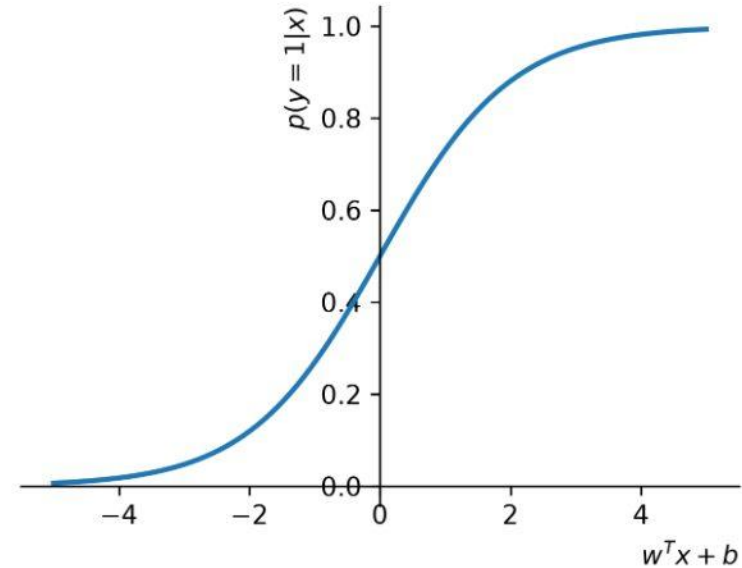
Logistic Regression

$$\log\left(\frac{p(y = 1|\mathbf{x})}{p(y = -1|\mathbf{x})}\right) = \mathbf{w}^T \mathbf{x} + b$$

$$p(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x} - b}}$$

$$\min_{\mathbf{w} \in \mathbb{R}^p, b \in \mathbb{R}} \sum_{i=1}^n \log(\exp(-y_i(\mathbf{w}^T \mathbf{x}_i + b)) + 1)$$

$$\hat{y} = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$$



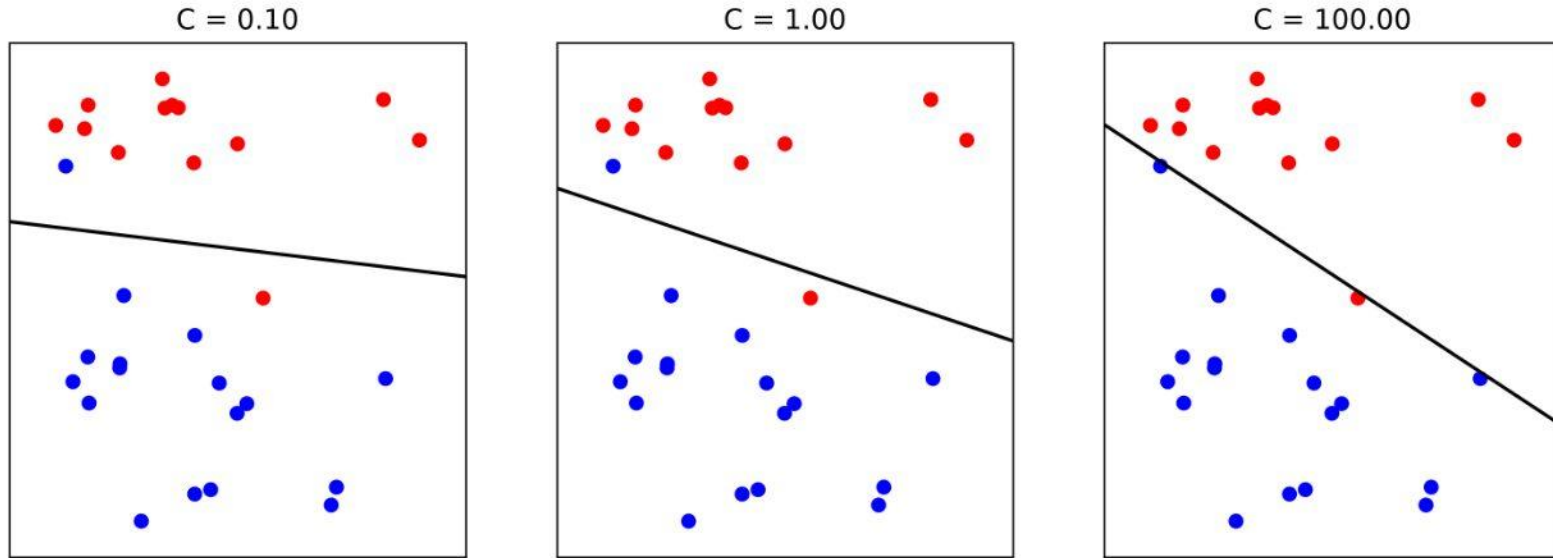
Penalized Logistic Regression

$$\min_{w \in \mathbb{R}^p, b \in \mathbb{R}} C \sum_{i=1}^n \log(\exp(-y_i(w^T \mathbf{x}_i + b)) + 1) + \frac{1}{2} \|w\|_2^2$$

$$\min_{w \in \mathbb{R}^p, b \in \mathbb{R}} C \sum_{i=1}^n \log(\exp(-y_i(w^T \mathbf{x}_i + b)) + 1) + \|w\|_1$$

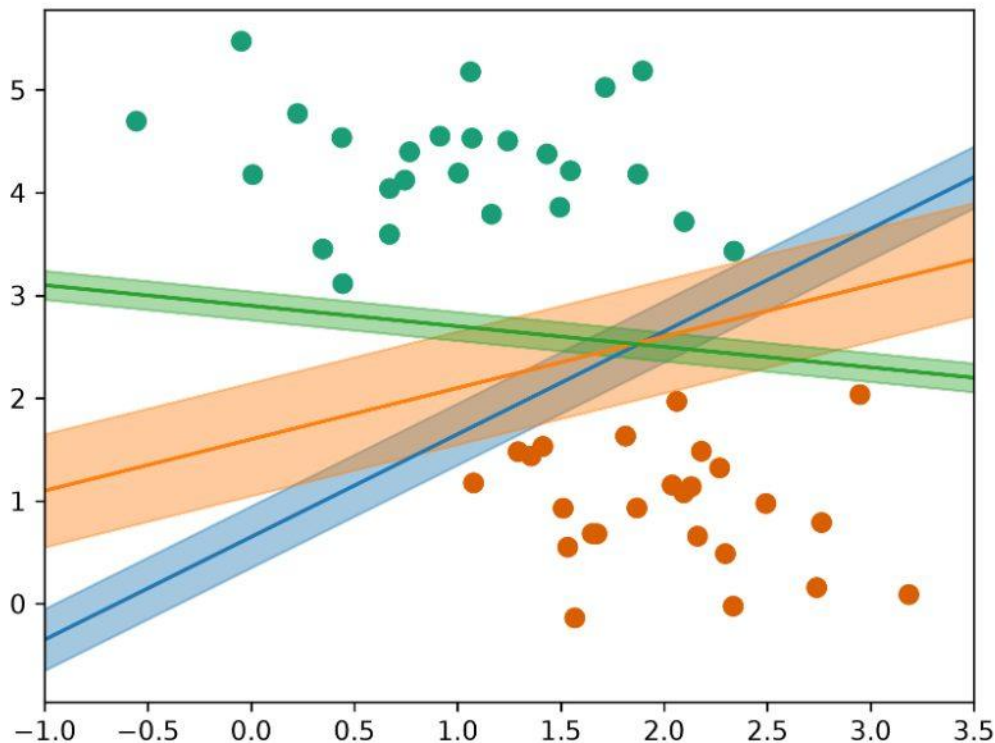
- C is inverse to alpha: small C = strong regularization
- scikit-learn: `LogisticRegression(C=..., l1_ratio=...)`; `l1_ratio=0` is L2 (default), `1` is L1, in between is elastic net; `C=np.inf` means no penalty

Effect of regularization



- Small C (a lot of regularization) limits the influence of individual points!

Max-Margin and Support Vectors



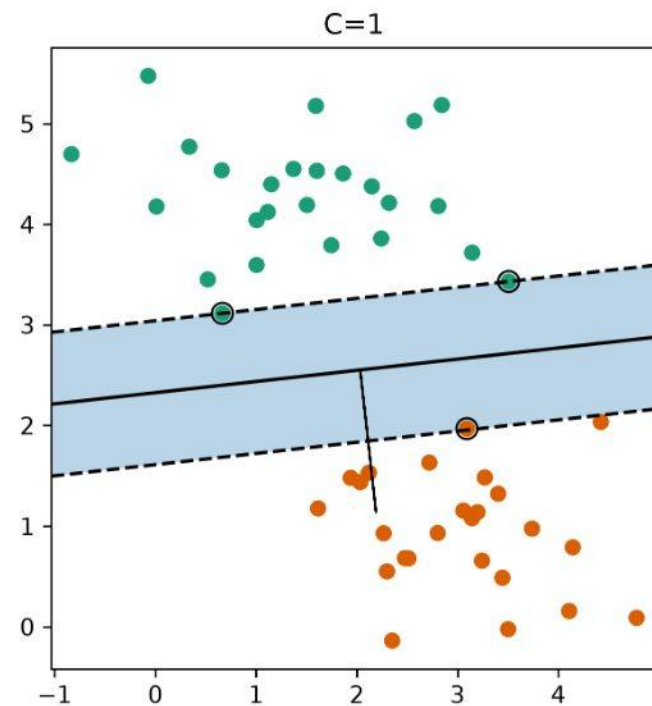
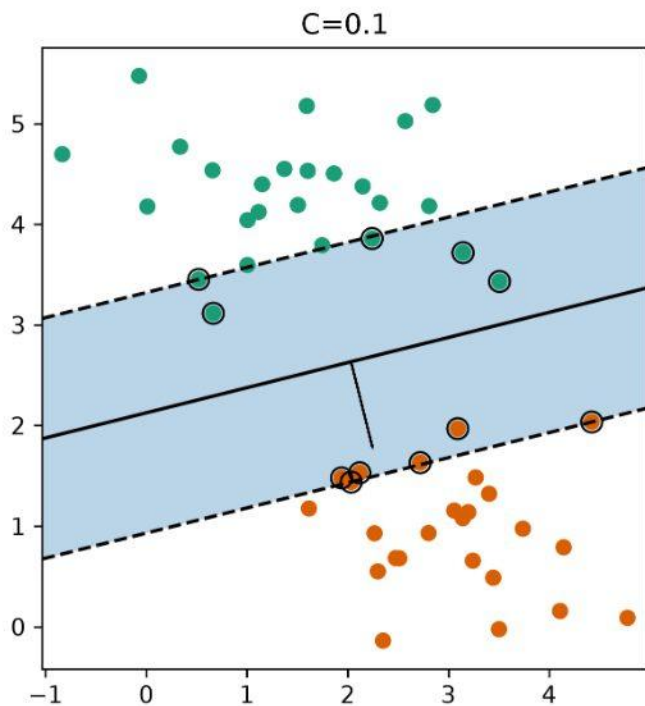
(soft margin) linear SVM

$$\min_{w \in \mathbb{R}^p, b \in \mathbb{R}} C \sum_{i=1}^n \max(0, 1 - y_i(w^T \mathbf{x}_i + b)) + ||w||_2^2$$

$$\min_{w \in \mathbb{R}^p, b \in \mathbb{R}} C \sum_{i=1}^n \max(0, 1 - y_i(w^T \mathbf{x}_i + b)) + ||w||_1$$

Within margin $\Leftrightarrow y_i(w^T \mathbf{x}_i + b) < 1$ Smaller $w \Rightarrow$ larger margin

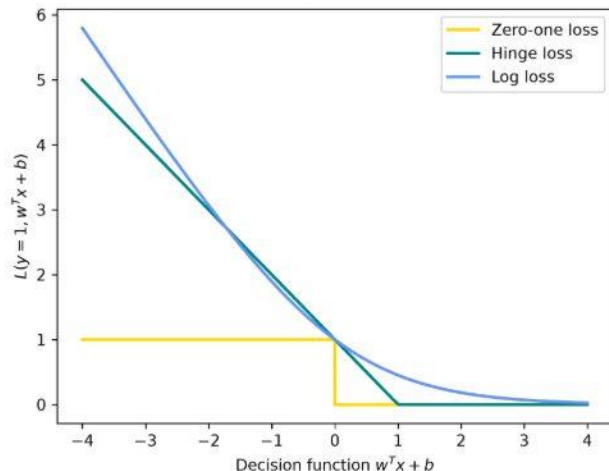
Max-Margin and Support Vectors



Logistic Regression vs SVM

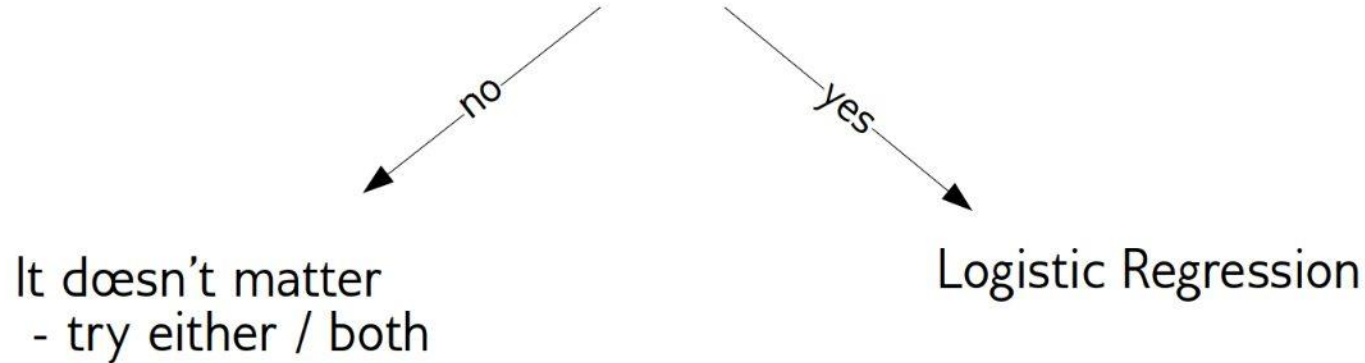
$$\min_{w \in \mathbb{R}^p, b \in \mathbb{R}} C \sum_{i=1}^n \log(\exp(-y_i(w^T \mathbf{x}_i + b)) + 1) + ||w||_2^2$$

$$\min_{w \in \mathbb{R}^p, b \in \mathbb{R}} C \sum_{i=1}^n \max(0, 1 - y_i(w^T \mathbf{x}_i + b)) + ||w||_2^2$$



SVM or LogReg?

Do you need probability estimates?



- Need compact model or believe solution is sparse? Use L1

Multiclass classification

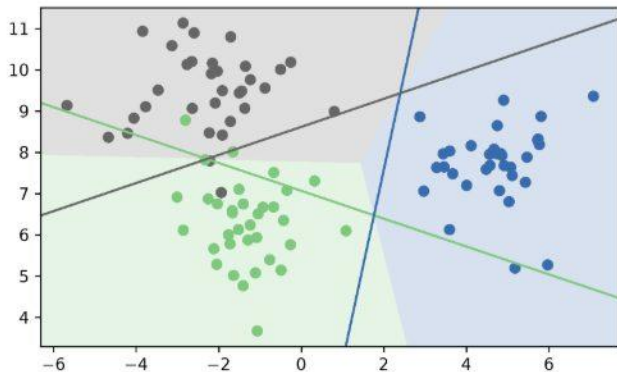
One vs Rest (OvR)

- One binary classifier per class: class i against all other classes
- For 4 classes: $1v\{2,3,4\}$, $2v\{1,3,4\}$, $3v\{1,2,4\}$, $4v\{1,2,3\}$
- Each trained on all the data
- Predict the class with the highest score:

$$\hat{y} = \arg \max_{i \in Y} \mathbf{w}_i^T \mathbf{x} + b_i$$

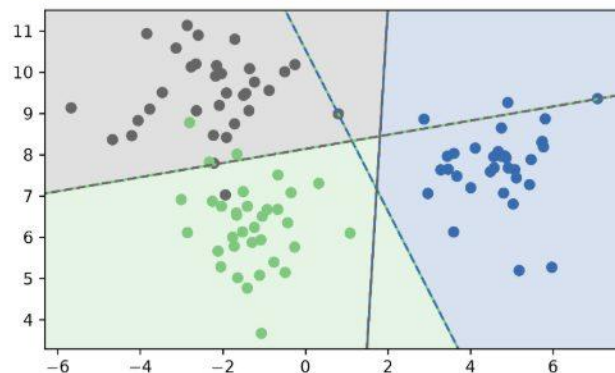
One vs Rest

- n_{classes} classifiers
- trained on imbalanced datasets of original size
- Retains some uncertainty?



One vs One

- $n_{\text{classes}} * (n_{\text{classes}} - 1) / 2$ classifiers
- trained on balanced subsets
- No uncertainty propagated



Multinomial Logistic Regression

Probabilistic multi-class model:

$$p(y = i|x) = \frac{e^{\mathbf{w}_i^T \mathbf{x} + b_i}}{\sum_{j=1}^k e^{\mathbf{w}_j^T \mathbf{x} + b_j}}$$

$$\min_{\mathbf{w} \in \mathbb{R}^{pk}, \mathbf{b} \in \mathbb{R}^k} - \sum_{i=1}^n \log(p(y = y_i | \mathbf{x}_i, \mathbf{w}, \mathbf{b}))$$

$$\hat{y} = \arg \max_{i=1, \dots, k} \mathbf{w}_i^T \mathbf{x} + b_i$$

- Same prediction rule as OvR !

In scikit-learn

- `LogisticRegression`: multinomial (softmax) for more than two classes; the `multi_class` option was removed
- `LinearSVC`: one-vs-rest; `SVC`: one-vs-one internally
- Explicit reductions for any estimator: `OneVsRestClassifier`, `OneVsOneClassifier`
- `clf.decision_function` = $w^T x + b$
- `logreg.predict_proba`
- `SVC(probability=True)`: slow, internal cross-validation, rarely worth it

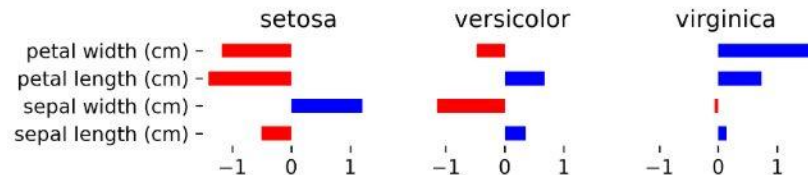
Multi-Class in Practice

```
iris = load_iris()
logreg = LogisticRegression(max_iter=1000).fit(iris.data, iris.target)
linearsvm = LinearSVC().fit(iris.data, iris.target)
print(logreg.coef_.shape, linearsvm.coef_.shape)
```

(3, 4) (3, 4)

logreg.coef_

```
array([[ -0.42 ,  0.967, -2.519, -1.084],
       [ 0.531, -0.315, -0.203, -0.943],
       [-0.112, -0.652,  2.722,  2.028]])
```



(after scaling the data, without intercept)

Kernel SVMs

Motivation

- Go from linear models to more powerful nonlinear ones.
- Keep convexity (ease of optimization).
- Generalize the concept of feature engineering.

Introducing Kernels

$$\hat{y} = \text{sign} \left(\sum_i^n \alpha_i (\mathbf{x}_i^T \mathbf{x}) \right) \rightarrow \hat{y} = \text{sign} \left(\sum_i^n \alpha_i (\phi(\mathbf{x}_i)^T \phi(\mathbf{x})) \right)$$

$$\phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j) \rightarrow k(\mathbf{x}_i, \mathbf{x}_j)$$

k positive definite, symmetric $\Rightarrow$ there exists a ϕ ! (possibly ∞ -dim)

Kernels in Practice

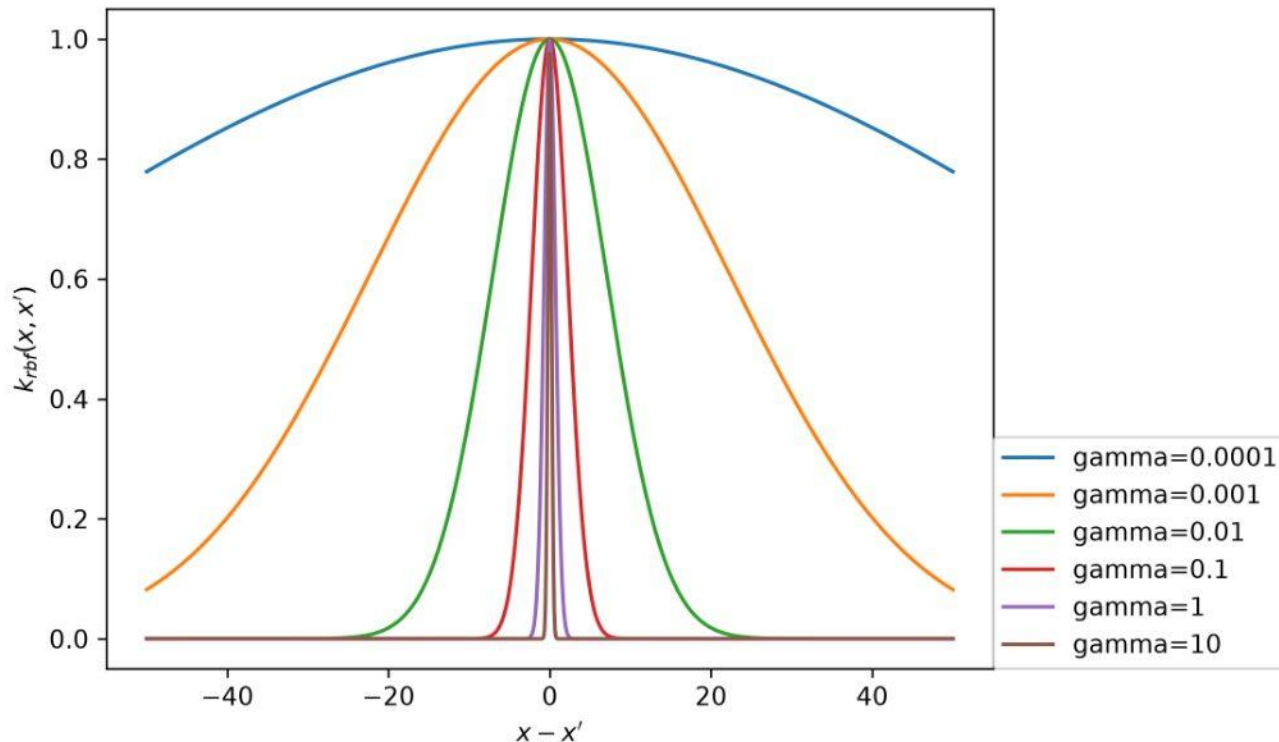
- Dual coefficients less interpretable
- Long runtime for "large" datasets (100k samples)
- Real power in infinite-dimensional spaces: rbf!
- Rbf is "universal kernel" - can learn (aka overfit) anything.
- Scale up with kernel approximation: Nystroem, RBFSampler + a linear model

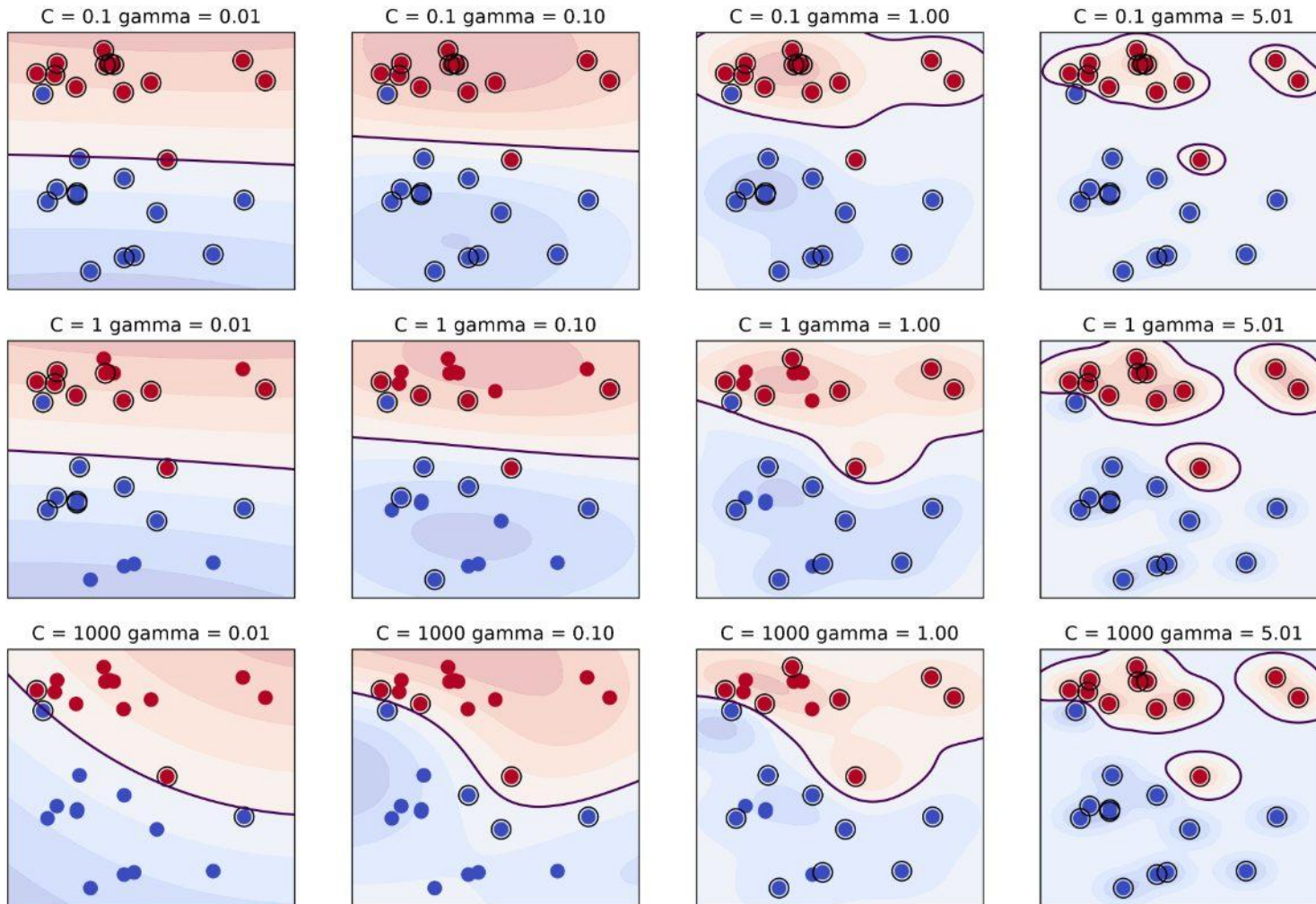
Preprocessing

- Kernels use inner products or distances.
- Scale your data: `StandardScaler` or `MinMaxScaler`, inside the pipeline
- Gamma parameter in RBF directly relates to scaling of data and `n_features` - the default `gamma='scale'` is `1/(X.var() * n_features)`

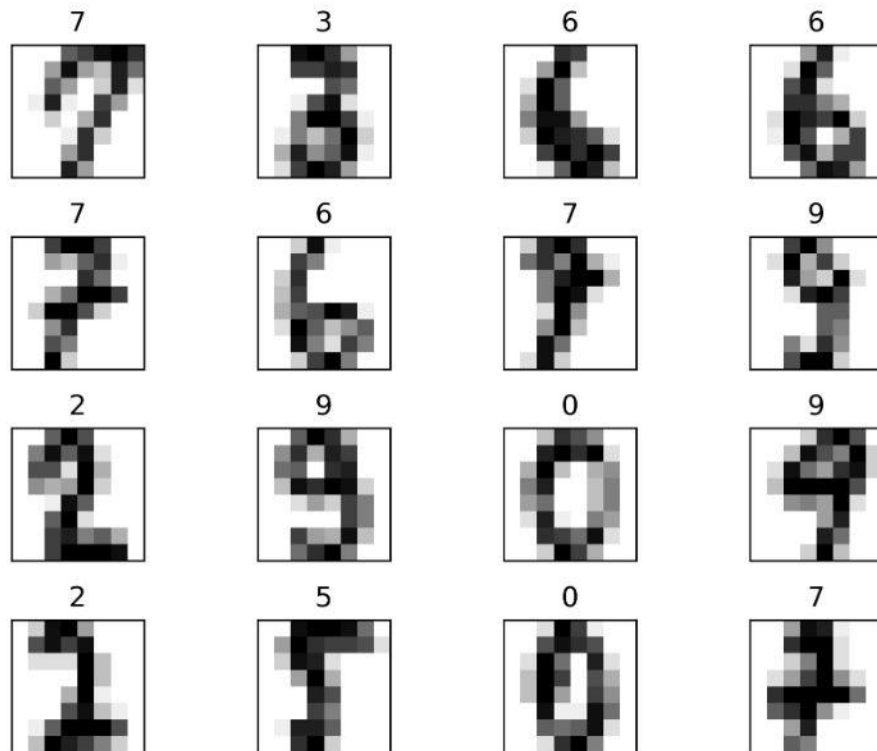
Parameters for RBF Kernels

- Regularization parameter C is limit on alphas (for any kernel)
- Gamma is bandwidth: $k_{\text{rbf}}(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$





```
from sklearn.datasets import load_digits
digits = load_digits()
```



Try it 2: predict, then run (10 min)

Breast cancer data, 10-fold CV accuracy on the training set. **Write down your ranking first:**

1. `LogisticRegression()` on raw features
2. `LogisticRegression()` after `StandardScaler`
3. `SVC()` on raw features
4. `SVC()` after `StandardScaler`

Then run section "Try it 2" in `lab.ipynb`. Which result surprised you? Tune `C` for (2): would you call its largest coefficients "causes of malignancy"?

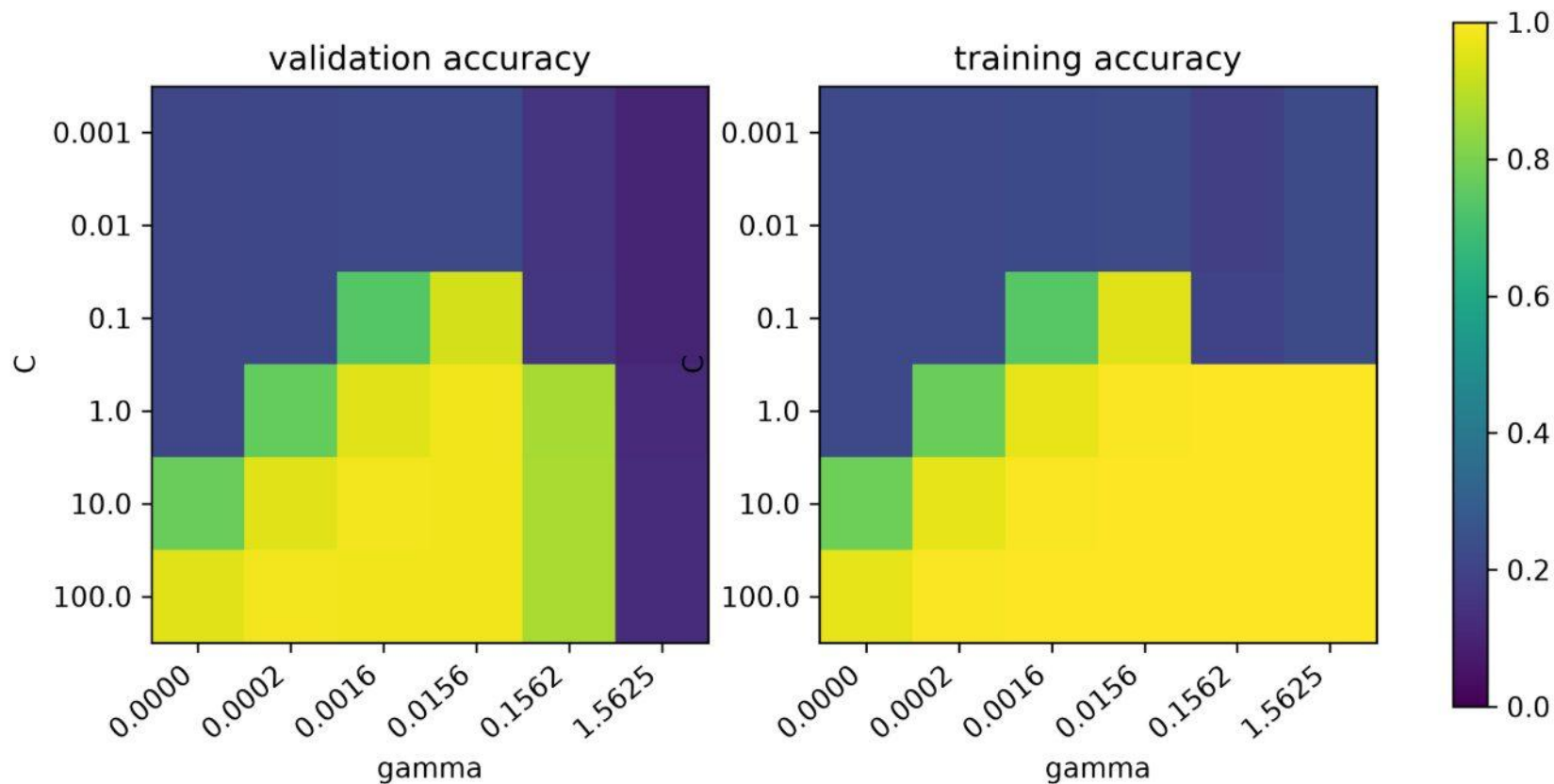
Grid-Searching Parameters

```
param_grid = {'svc__C': np.logspace(-3, 2, 6),  
              'svc__gamma': np.logspace(-3, 2, 6) / X_train.shape[1]}  
param_grid
```

```
{'svc__C': array([0.001, 0.01, 0.1, 1., 10., 100.]),  
 'svc__gamma': array([0.000016, 0.000156, 0.001563, 0.015625, 0.15625, 1.5625])}
```

```
grid = GridSearchCV(scaled_svc, param_grid=param_grid, cv=10, return_  
grid.fit(X_train, y_train)
```

Grid-Searching Parameters



Comparing models in MLflow

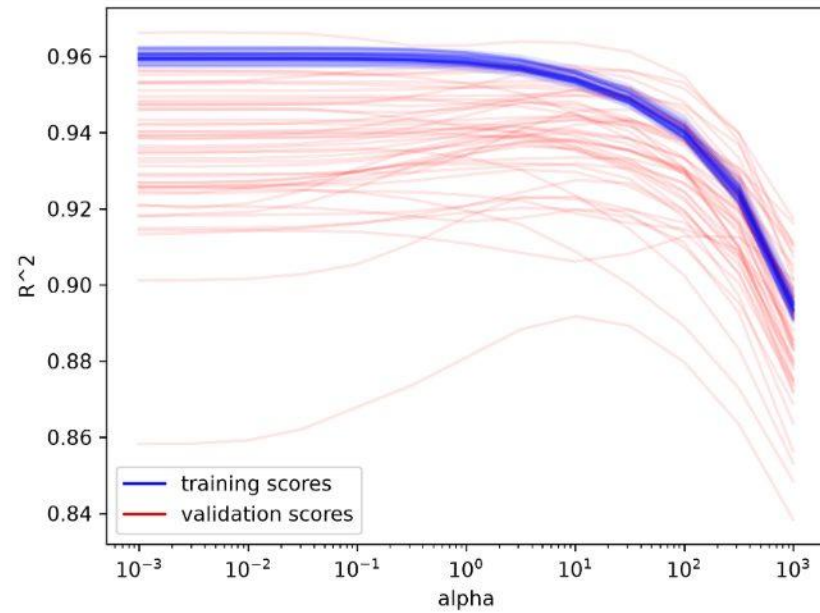
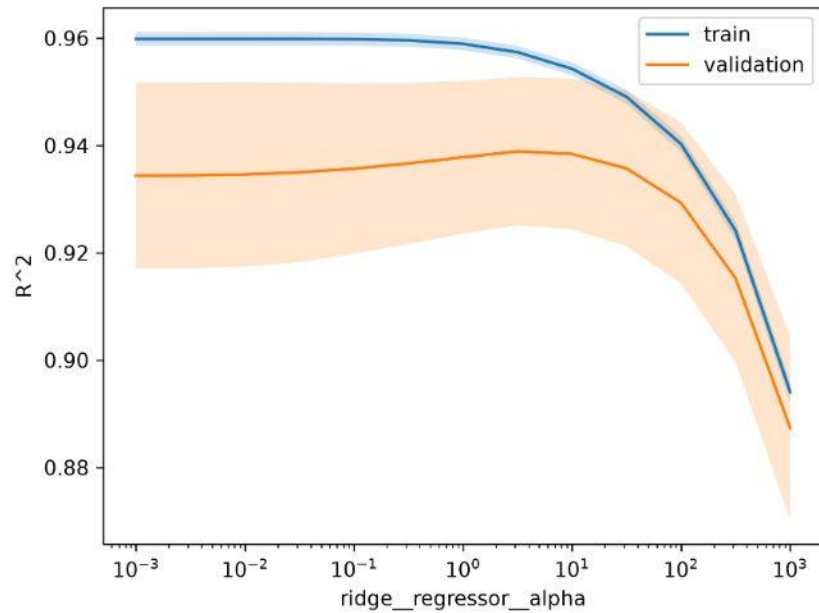
```
with mlflow.start_run(run_name="digits-linear-vs-kernel"):
    for name, model in models.items():
        scores = cross_val_score(model, X_train, y_train, cv=cv)
        with mlflow.start_run(run_name=name, nested=True):
            mlflow.log_param("model", name)
            mlflow.log_metrics({"cv_accuracy_mean": scores.mean(),
                               "cv_accuracy_std": scores.std(), "cv_seconds": elap
```

logreg_C1	0.973 +/- 0.013
logreg_C0.01	0.943 +/- 0.015
linear_svc_C1	0.957 +/- 0.020
svc_rbf_default	0.980 +/- 0.011
svc_rbf_tuned	0.982 +/- 0.007

Summary

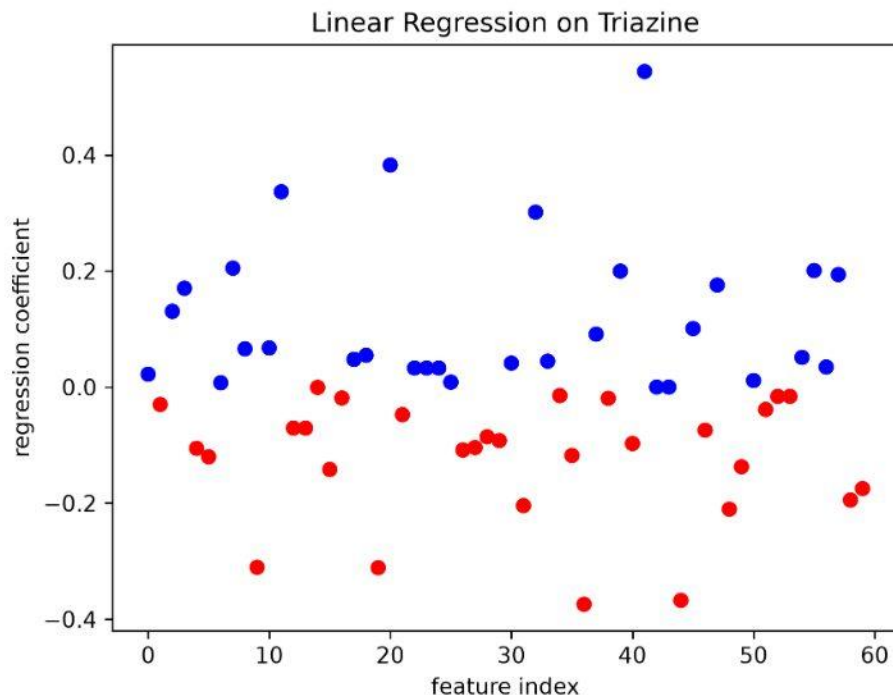
- Linear models: same prediction rule, different loss and penalty
- Ridge shrinks, Lasso selects, Elastic Net does both; tune α / C on a log grid
- Scale features for any penalized or kernel model; log-transform skewed targets
- Logistic regression when you need probabilities; linear SVM otherwise comparable
- Kernel SVMs: non-linear and convex, but scale poorly with n_{samples}
- Next week: trees, random forests and gradient boosting

Backup



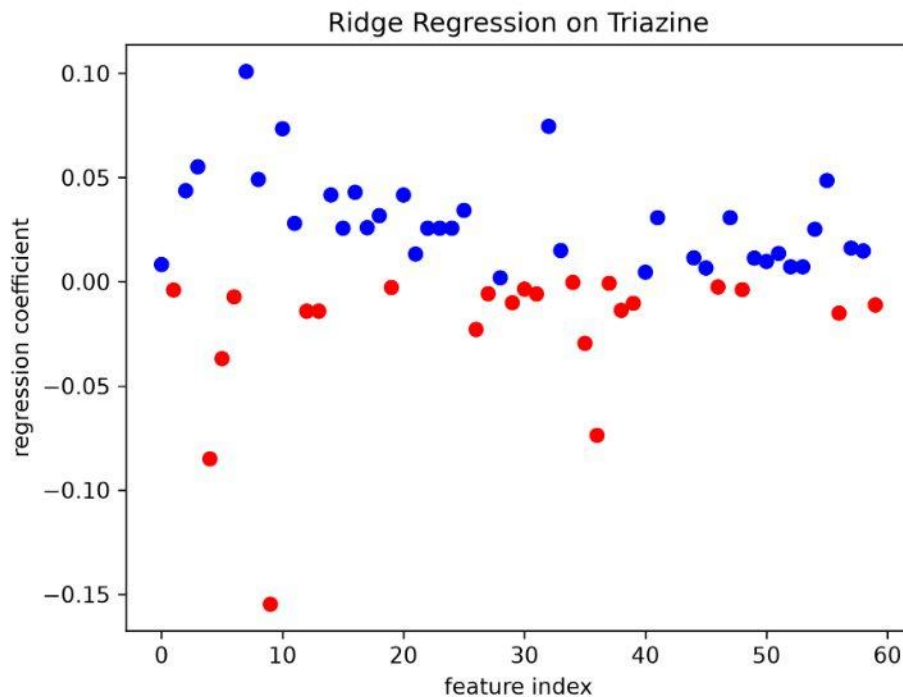
Plotting coefficient values (LR)

```
lr = LinearRegression().fit(X_train, y_train)
plt.scatter(range(X_train.shape[1]), lr.coef_,
            c=np.sign(lr.coef_), cmap='bwr_r')
```

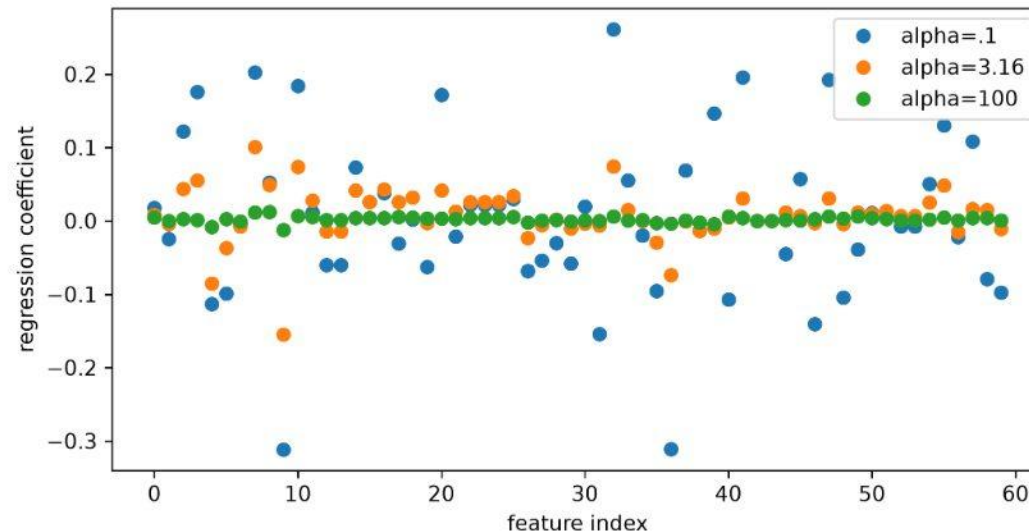


Ridge Coefficients

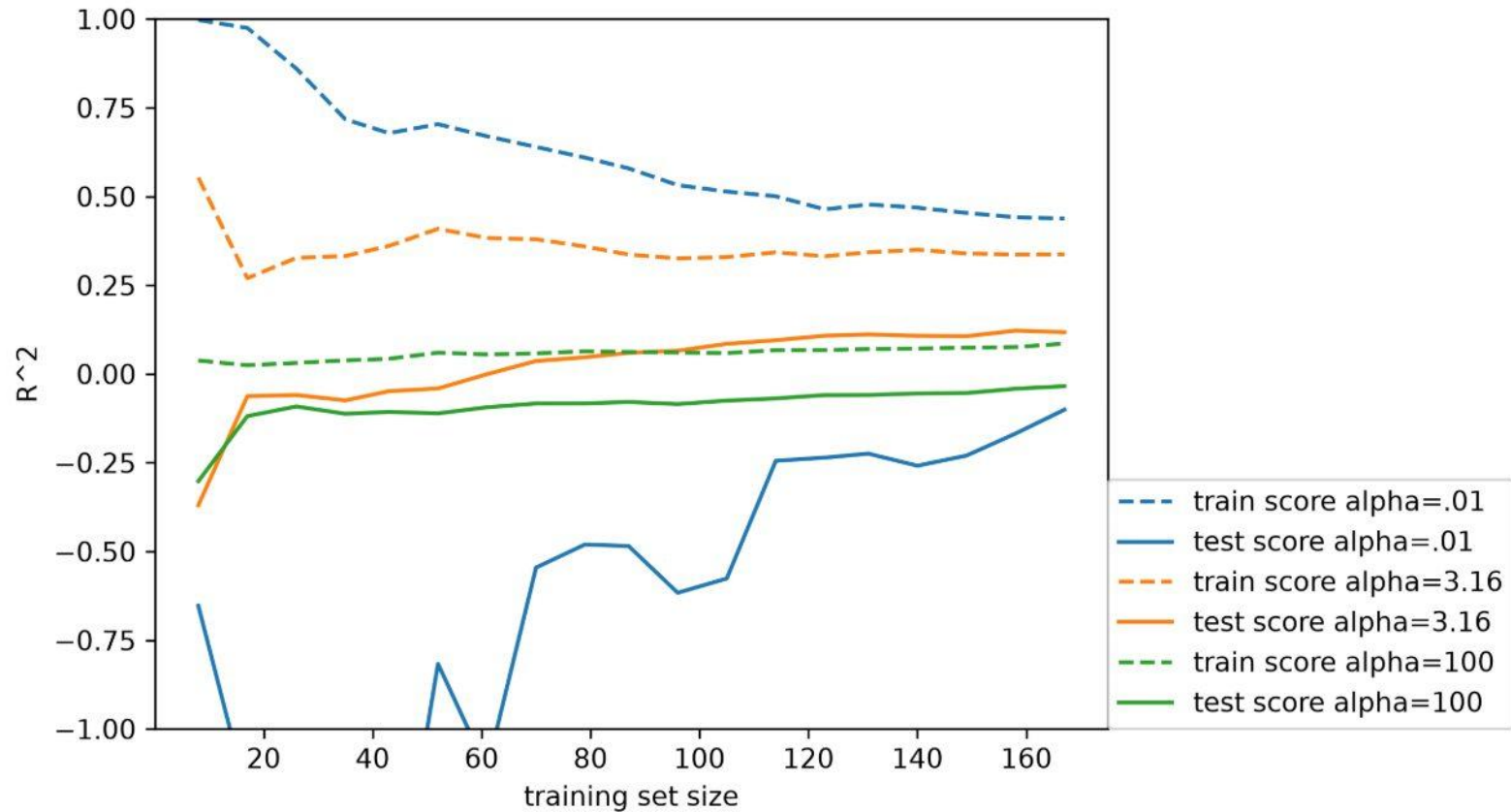
```
ridge = grid.best_estimator_  
plt.scatter(range(X_train.shape[1]), ridge.coef_,  
            c=np.sign(ridge.coef_), cmap="bwr_r")
```



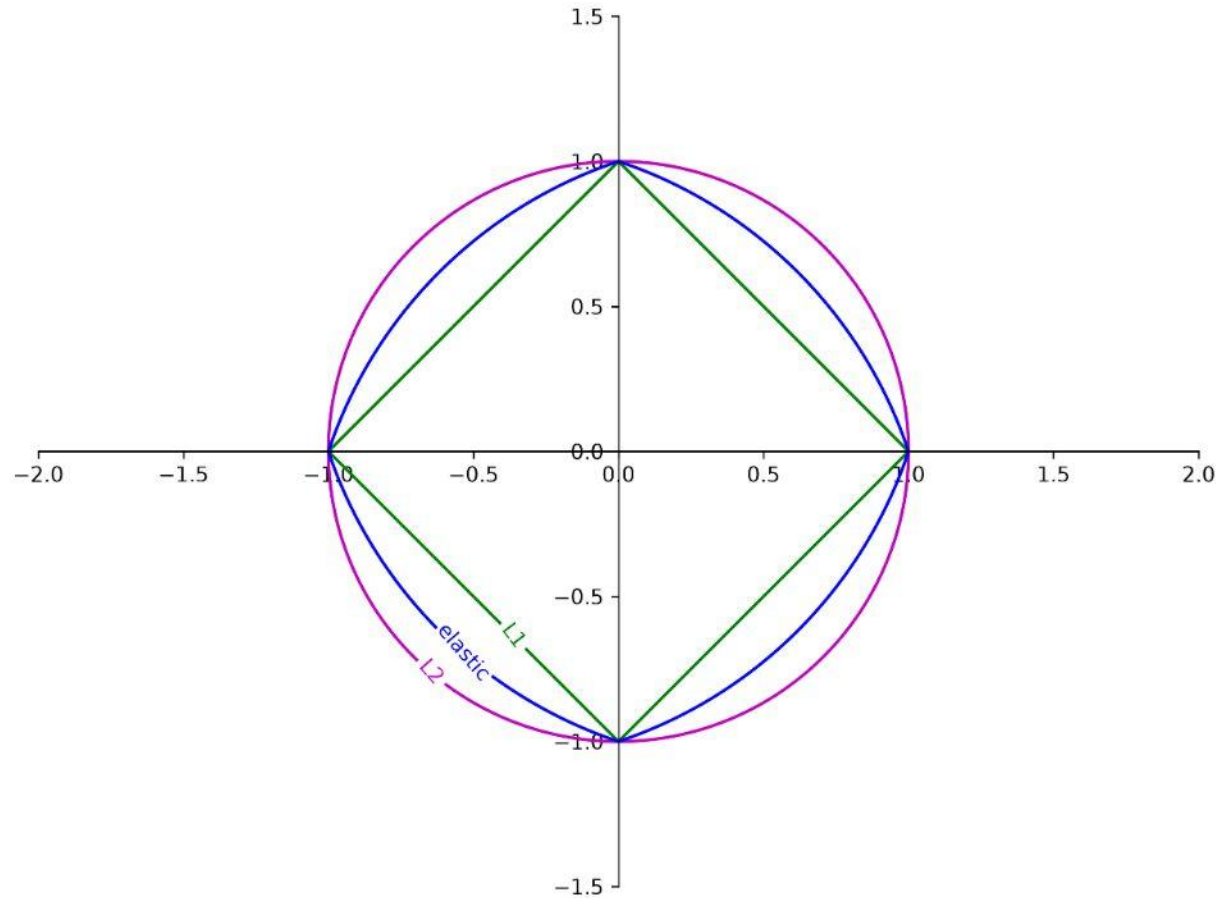
```
ridge100 = Ridge(alpha=100).fit(X_train, y_train)
ridge01 = Ridge(alpha=.1).fit(X_train, y_train)
plt.figure(figsize=(8, 4))
plt.plot(ridge01.coef_, 'o', label="alpha=.1")
plt.plot(ridge.coef_, 'o', label=f"alpha={ridge.alpha:.2f}")
plt.plot(ridge100.coef_, 'o', label="alpha=100")
plt.legend()
```



Learning Curves



Comparing unit balls



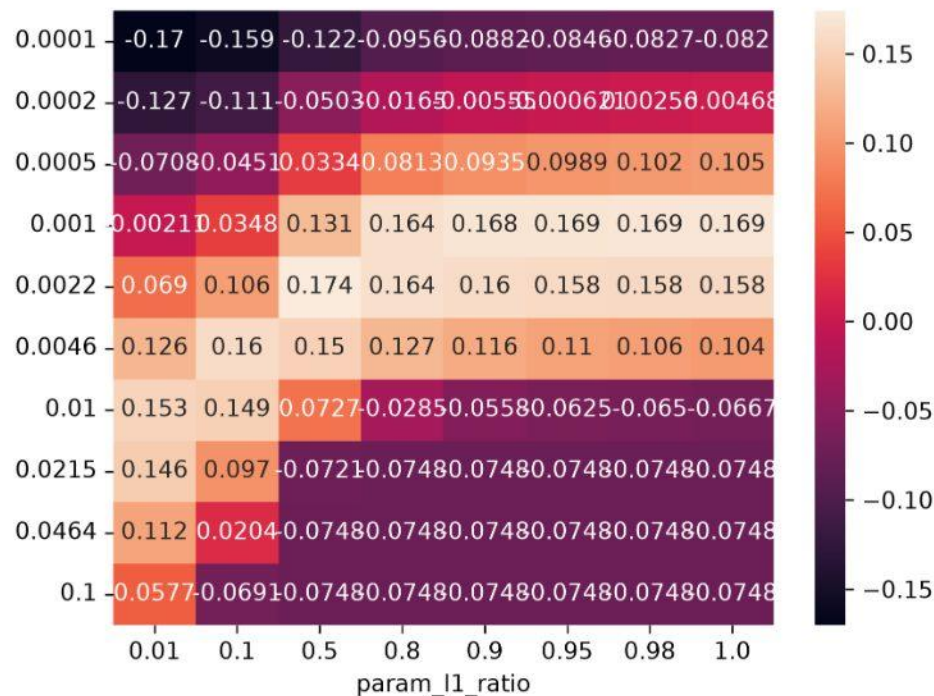
Parametrization in scikit-learn

$$\min_{w,b} \frac{1}{2n} \sum_{i=1}^n (w^T \mathbf{x}_i + b - y_i)^2 + \alpha \eta ||w||_1 + \frac{\alpha(1 - \eta)}{2} ||w||_2^2$$

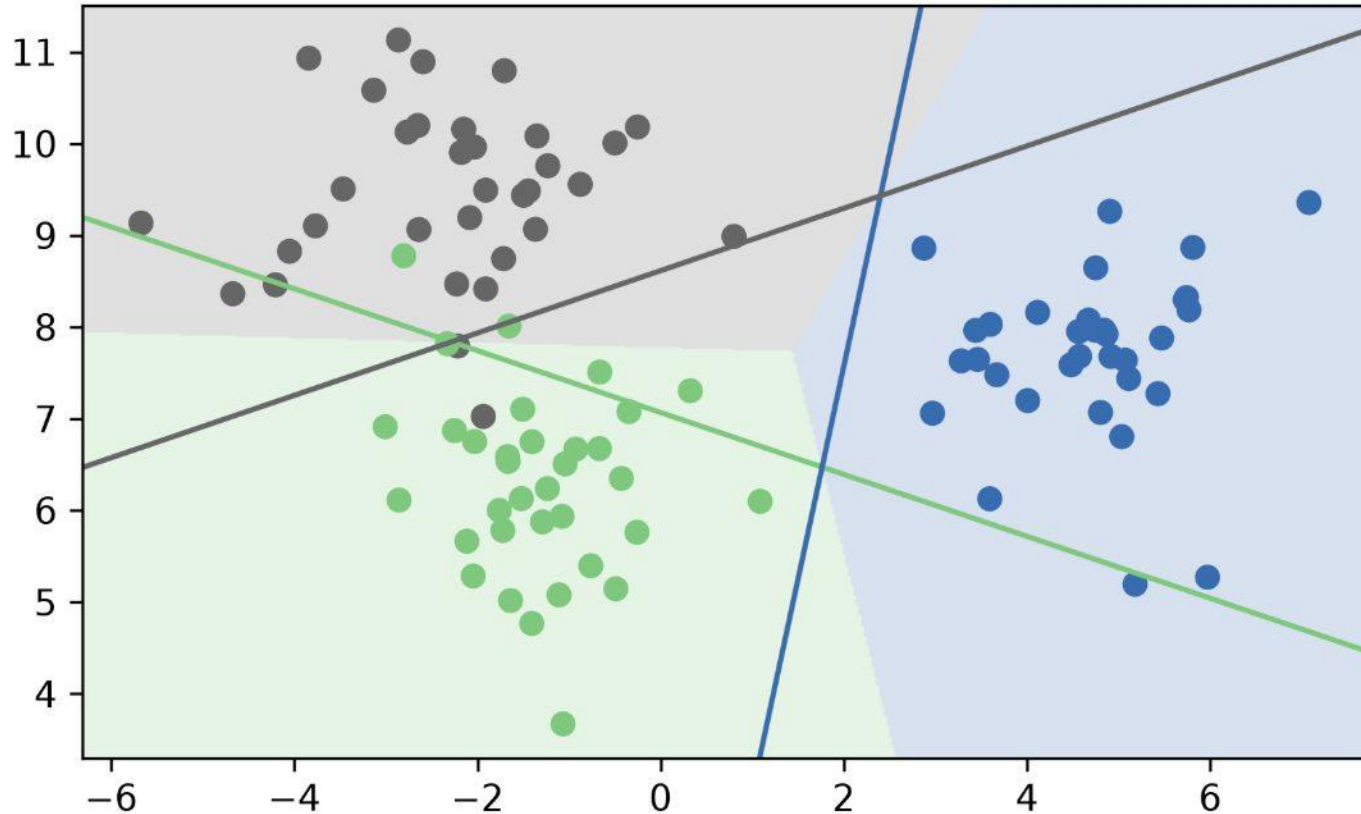
Where η is the relative amount of l1 penalty (`l1_ratio` in the code).

Analyzing grid-search results

```
import pandas as pd
res = pd.pivot_table(pd.DataFrame(grid.cv_results_),
                     values='mean_test_score', index='param_alpha', columns='param_l1_ratio')
```



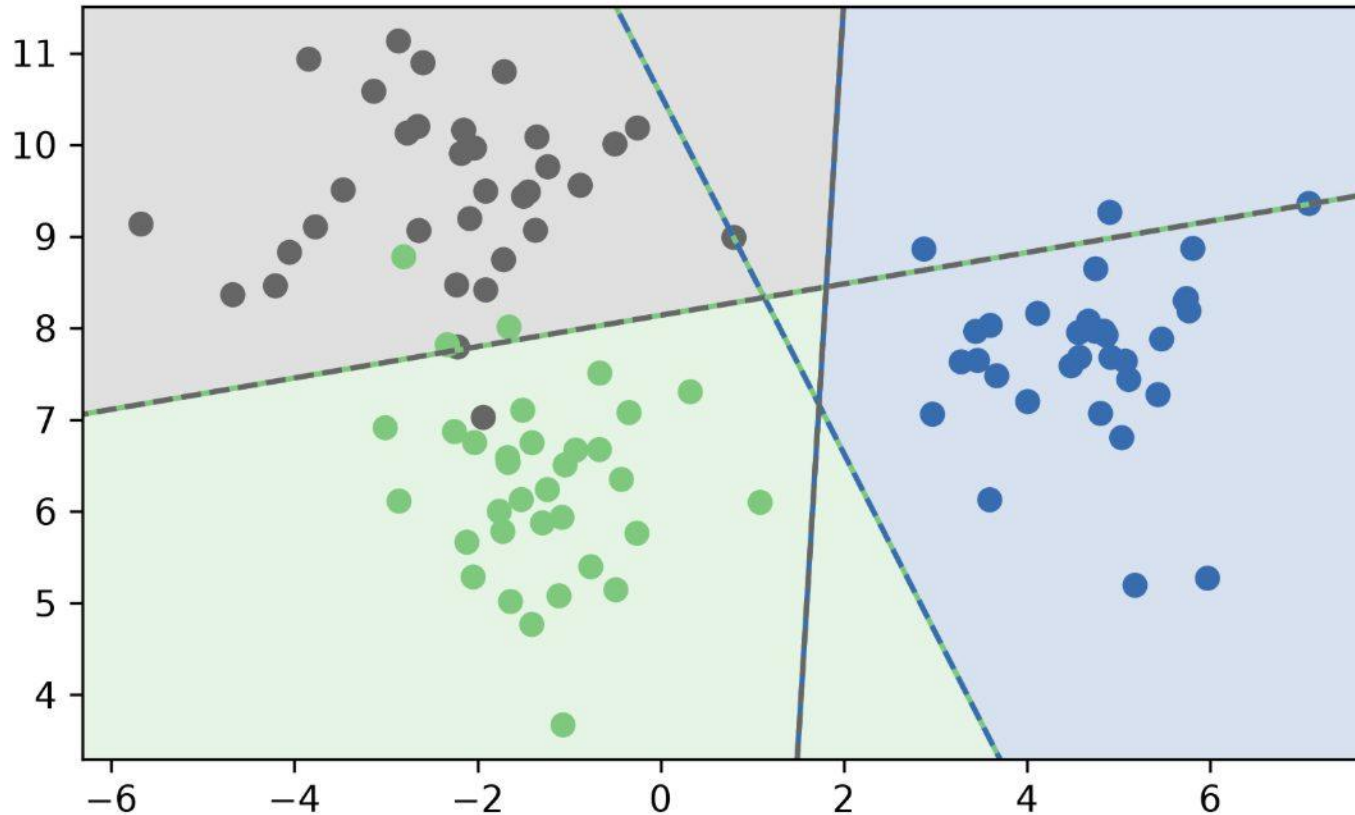
One vs Rest Prediction



One vs One (OvO)

- One binary classifier per pair of classes: 1v2, 1v3, 1v4, 2v3, 2v4, 3v4
- $n * (n-1) / 2$ classifiers, each trained only on the two classes involved
- Predict: every classifier votes, the class with most votes wins
- Again, a heuristic

One vs One Prediction



Computational considerations (all linear models)

- Don't use `SVC(kernel='linear')`, use `LinearSVC` (or `LogisticRegression`)
- `LinearSVC(dual='auto')` (default) picks primal or dual from the data shape
- `LogisticRegression`: default `lbfgs` is fine for small and medium data; `saga` for L1 / elastic net and large sparse data
- For `n_features >> n_samples` with L1: `LassoLars` / `lars_path`
- For small `n_samples` (< 10,000), don't worry
- `SGDClassifier` / `SGDRegressor` for data that does not fit in memory (`partial_fit`)

Reformulate Linear Models

- Optimization Theory

$$\mathbf{w} = \sum_{i=1}^n \alpha_i \mathbf{x}_i$$

(alpha are dual coefficients. Non-zero for support vectors only)

$$\hat{y} = \text{sign}(\mathbf{w}^T \mathbf{x}) \Rightarrow \hat{y} = \text{sign} \left(\sum_i^n \alpha_i (\mathbf{x}_i^T \mathbf{x}) \right)$$

$$|\alpha_i| \leq C$$

Understanding Dual Coefficients

```
linear_svm.coef_
```

```
array([[0.139, 0.06, -0.201, 0.048, 0.019]])
```

$$y = \text{sign}(0.139x_0 + 0.06x_1 - 0.201x_0^2 + 0.048x_0x_1 + 0.019x_1^2)$$

```
linear_svm.dual_coef_  
#array([[-0.03, -0.003, 0.003, 0.03]])  
linear_svm.support_  
#array([1, 26, 42, 62], dtype=int32)
```

$$y = \text{sign}(-0.03\phi(\mathbf{x}_1)^T \phi(\mathbf{x}) - 0.003\phi(\mathbf{x}_{26})^T \phi(\mathbf{x}) + 0.003\phi(\mathbf{x}_{42})^T \phi(\mathbf{x}) + 0.03\phi(\mathbf{x}_{62})^T \phi(\mathbf{x}))$$

Scaling and Default Params

gamma : {'scale', 'auto'} or float, default='scale'
if gamma='scale' (default) is passed then it uses $1 / (n_features * X.var())$ as value of
if 'auto', uses $1 / n_features$

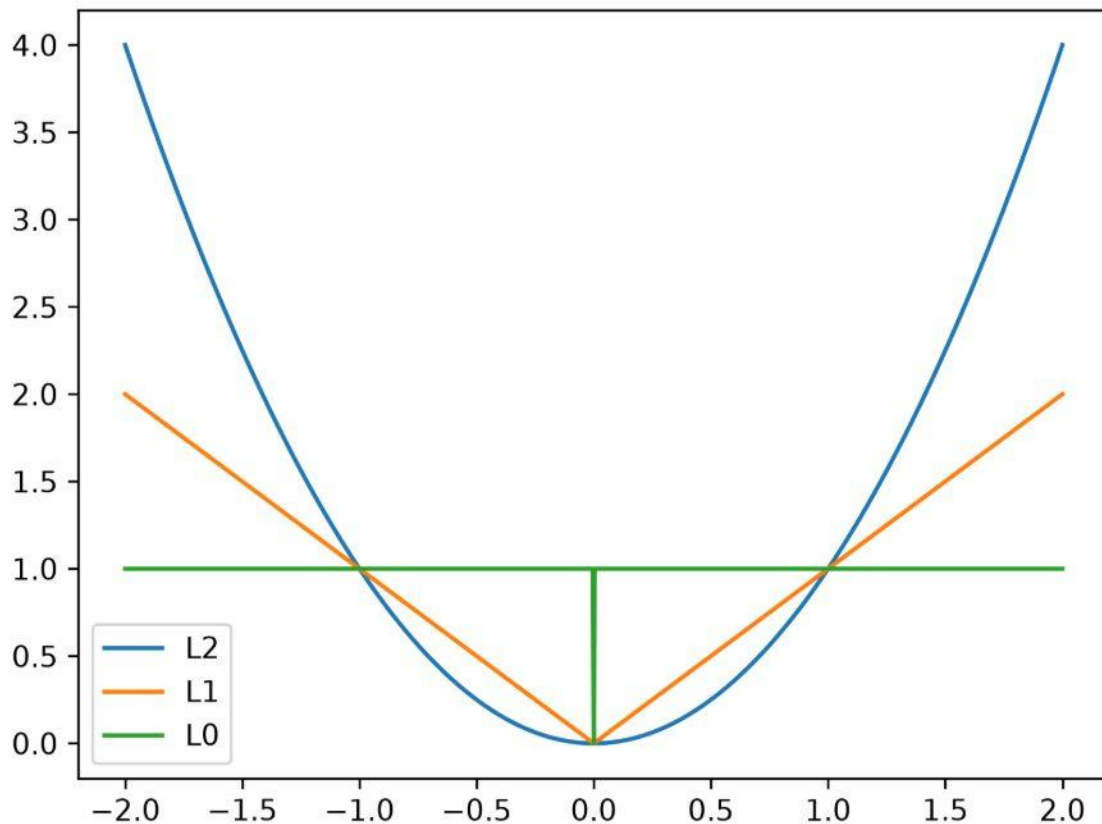
```
print('auto', np.mean(cross_val_score(SVC(gamma='auto'), X_train, y_train, cv=10)))  
print('scale', np.mean(cross_val_score(SVC(gamma='scale'), X_train, y_train, cv=10)))  
scaled_svc = make_pipeline(StandardScaler(), SVC())  
print('pipe', np.mean(cross_val_score(scaled_svc, X_train, y_train, cv=10)))
```

```
auto 0.563  
scale 0.987  
pipe 0.977
```

```
gamma = (1. / (X_train.shape[1] * X_train.var()))  
print(np.mean(cross_val_score(SVC(gamma=gamma), X_train, y_train, cv=10)))
```

```
0.987
```

Understanding L1 and L2 Penalties

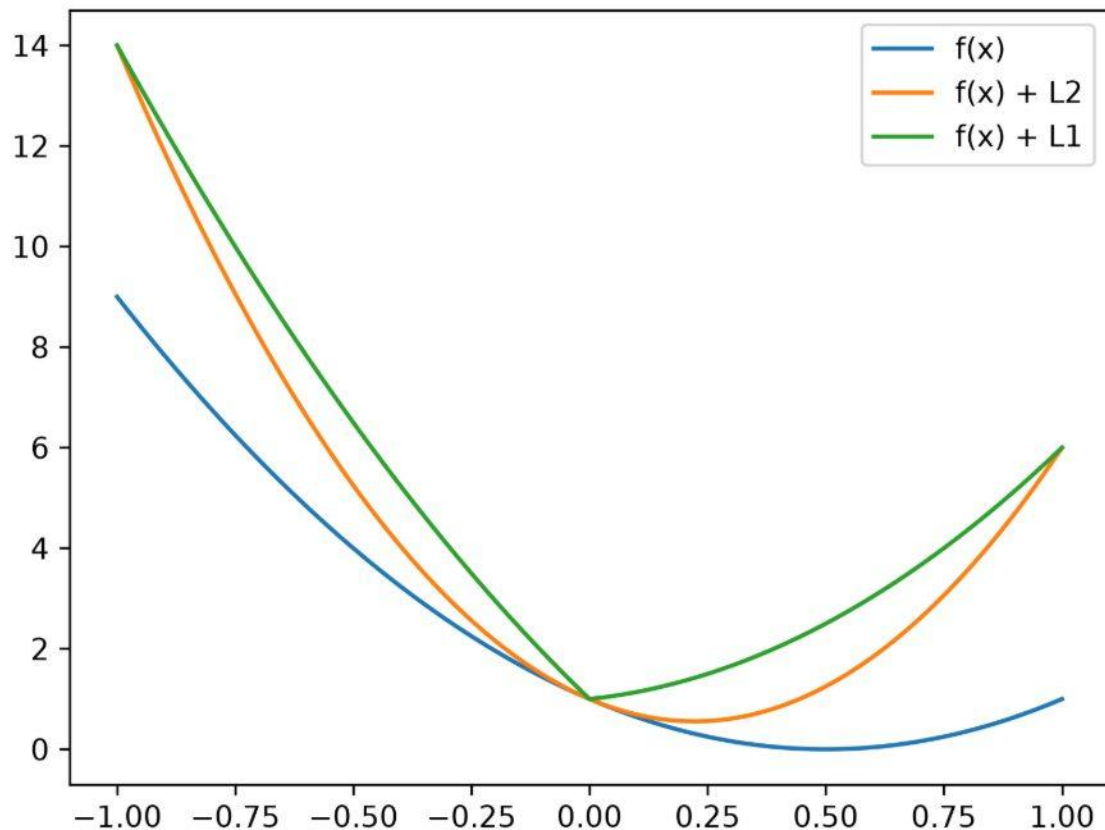


$$\ell_2(w) = \sqrt{\sum_i w_i^2}$$

$$\ell_1(w) = \sum_i |w_i|$$

$$\ell_0(w) = \sum_i 1_{w_i \neq 0}$$

Understanding L1 and L2 Penalties

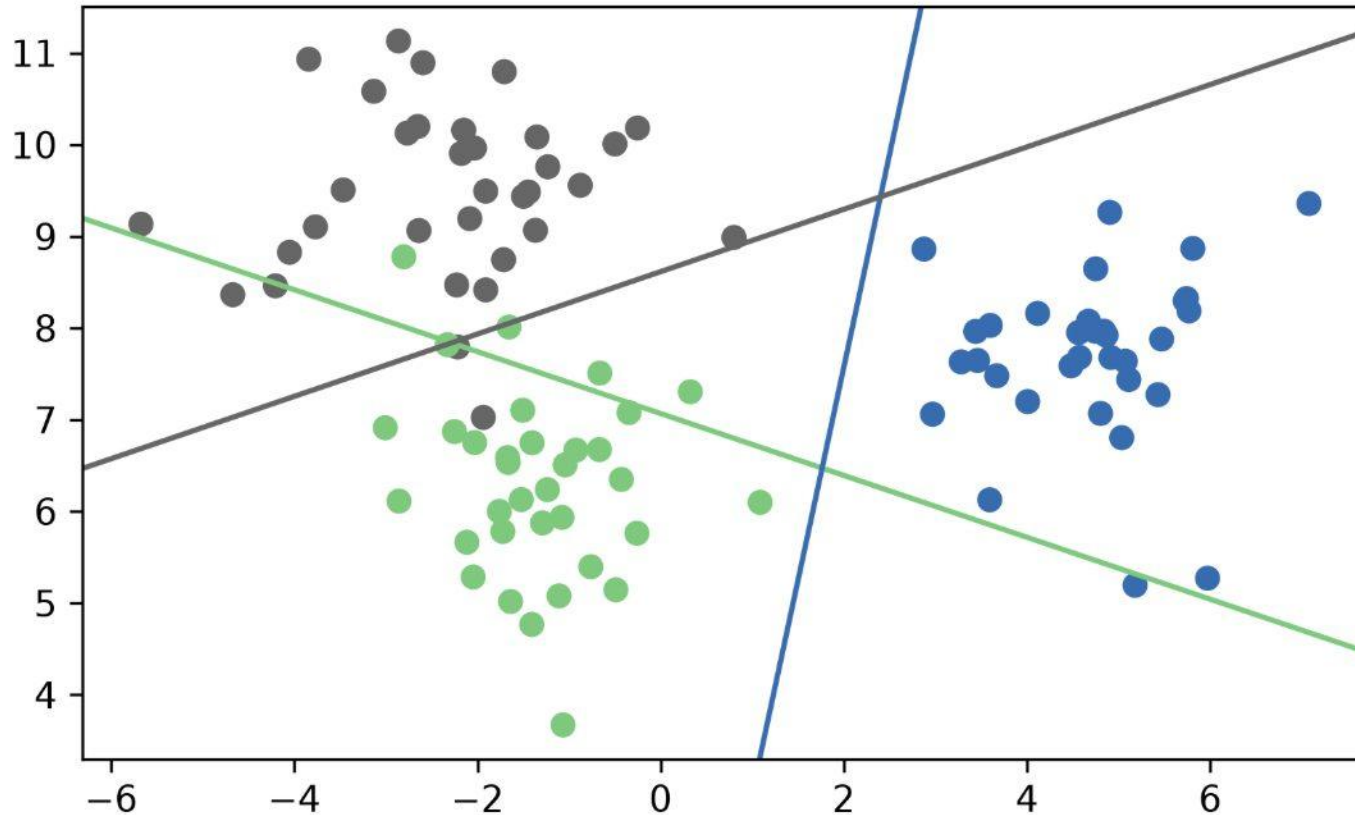


$$f(x) = (2x - 1)^2$$

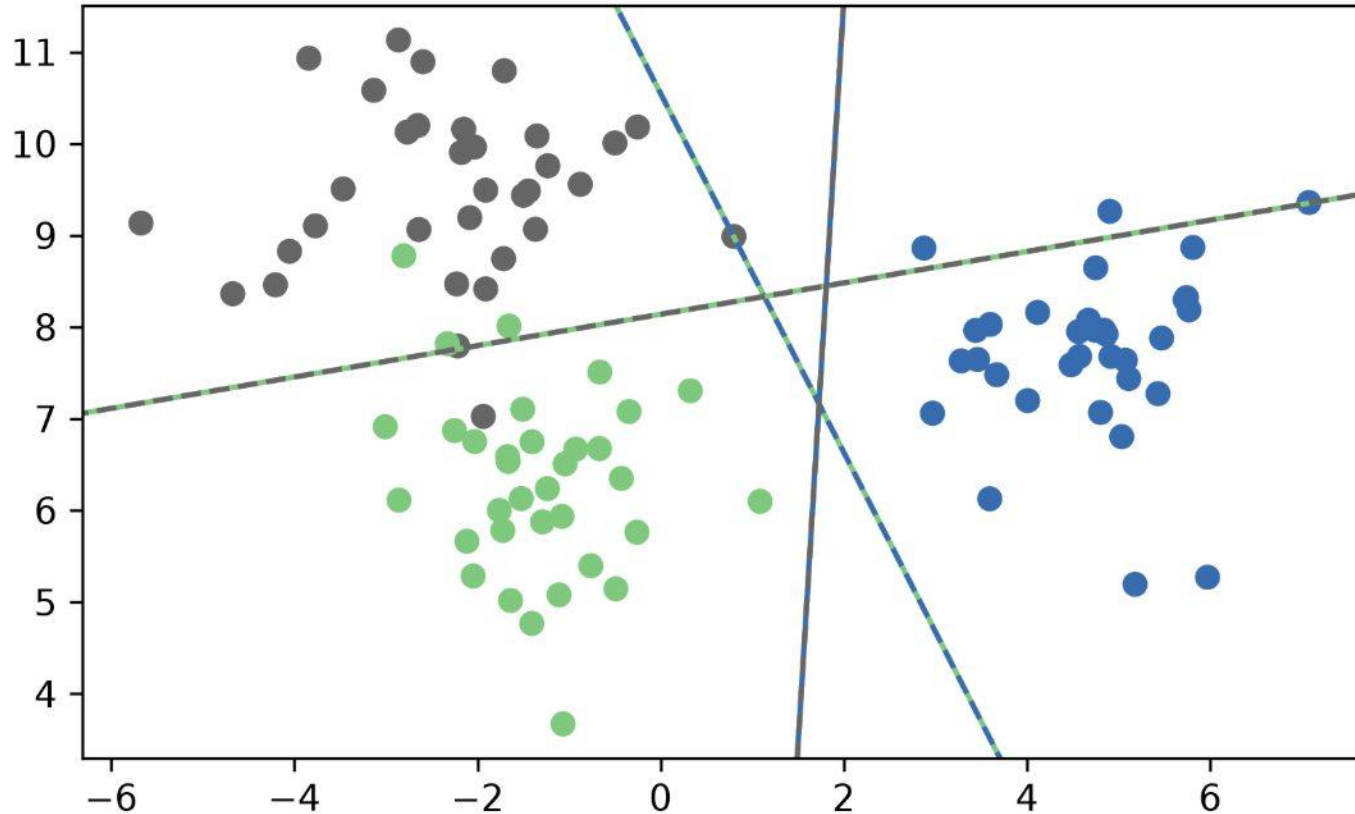
$$f(x) + L2 = (2x - 1)^2 + \alpha x^2$$

$$f(x) + L1 = (2x - 1)^2 + \alpha |x|$$

One vs Rest Prediction



One vs One Prediction



Examples of Kernels

$$k_{\text{linear}}(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}'$$

$$k_{\text{poly}}(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T \mathbf{x}' + c)^d$$

$$k_{\text{rbf}}(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$$

$$k_{\text{sigmoid}}(\mathbf{x}, \mathbf{x}') = \tanh(\gamma \mathbf{x}^T \mathbf{x}' + r)$$

$$k_{\cap}(\mathbf{x}, \mathbf{x}') = \sum_{i=1}^p \min(x_i, x'_i)$$

- If k and k' are kernels, so are $k + k'$, kk' , ck' , $\dots$

Polynomial Kernel vs Features

$$k_{\text{poly}}(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T \mathbf{x}' + c)^d$$

Primal vs Dual Optimization

Explicit polynomials → compute on `n_samples * n_features ** d`

Kernel trick → compute on kernel matrix of shape `n_samples * n_samples`

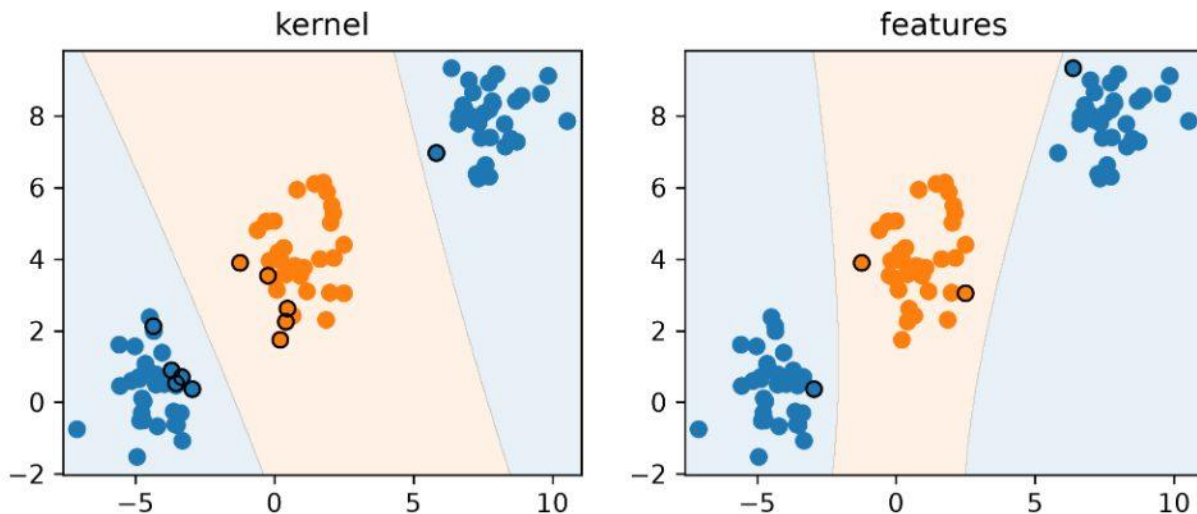
For a single feature:

$$(x^2, \sqrt{2}x, 1)^T (x'^2, \sqrt{2}x', 1) = x^2 x'^2 + 2xx' + 1 = (xx' + 1)^2$$

Poly kernels vs explicit features

```
poly = PolynomialFeatures(include_bias=False)
X_poly = poly.fit_transform(X)
print(X.shape, X_poly.shape)
print(poly.get_feature_names_out())
```

```
(100, 2) (100, 5)
['x0' 'x1' 'x0^2' 'x0 x1' 'x1^2']
```



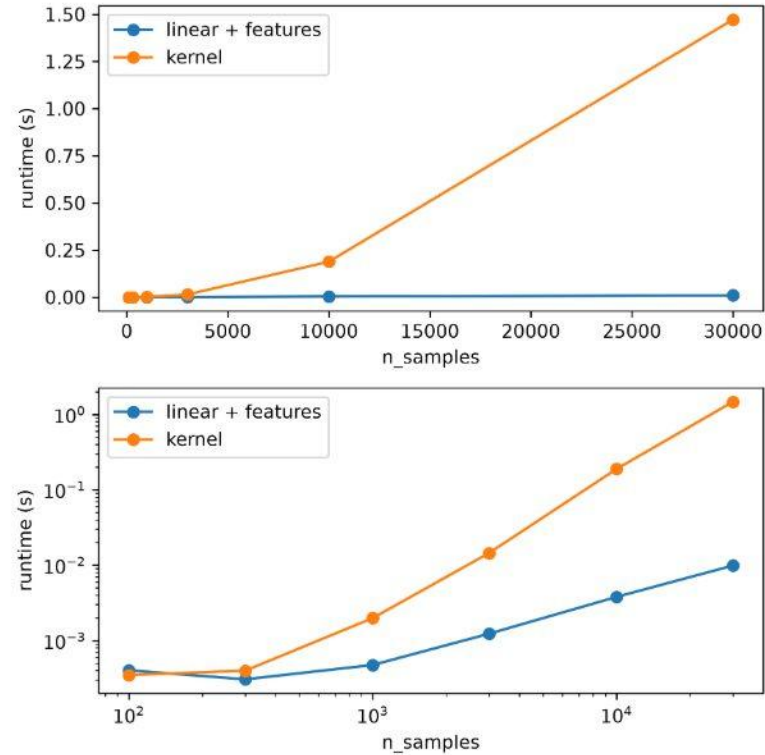
With Kernel

$$y = \text{sign} \left(\sum_i^n \alpha_i k(\mathbf{x}_i, \mathbf{x}) \right)$$

```
poly_svm.support_  
# array([ 1, 22, 41, 45, 61, 79,  9, 21, 23, 38, 42], dtype=int32)  
poly_svm.dual_coef_  
# array([[ -1., -0.304, -1., -0.307, -1., -1.,  0.612,  1.,  1.,  1.,  1.]])
```

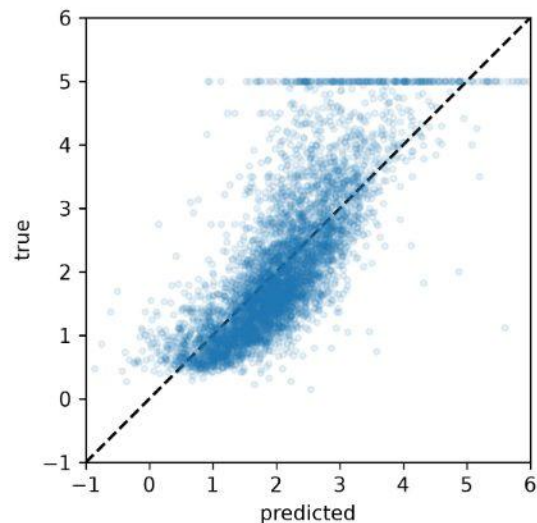
$$y = \text{sign} \left(-1 (\mathbf{x}_1^T \mathbf{x} + 1)^2 - 0.304 (\mathbf{x}_{22}^T \mathbf{x} + 1)^2 - \dots + 1 (\mathbf{x}_{42}^T \mathbf{x} + 1)^2 \right)$$

Runtime Considerations

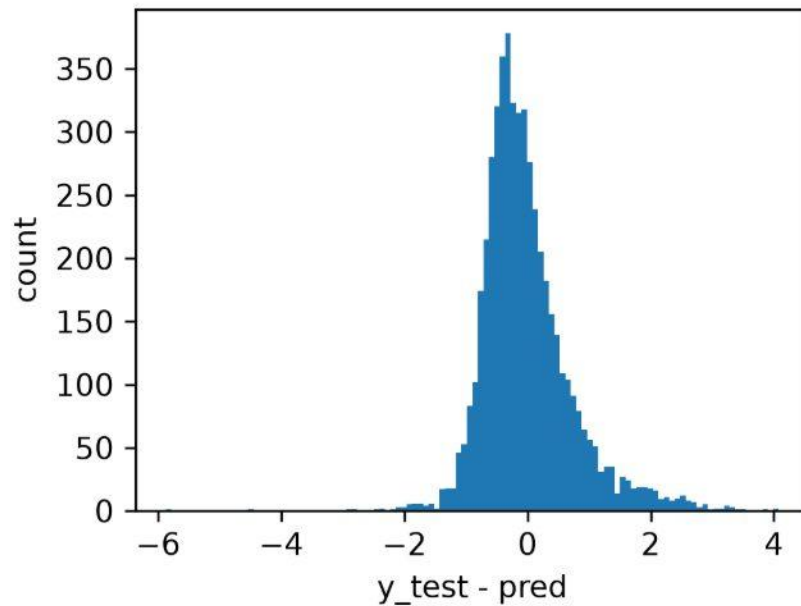
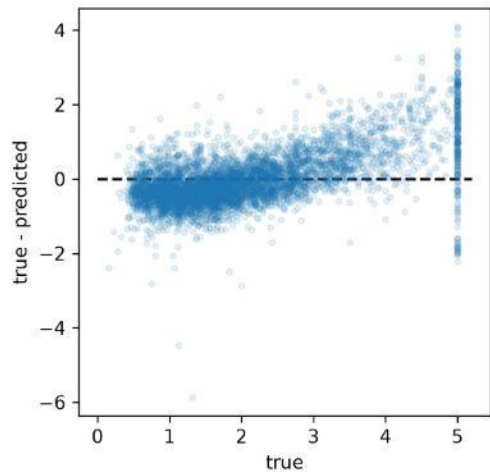
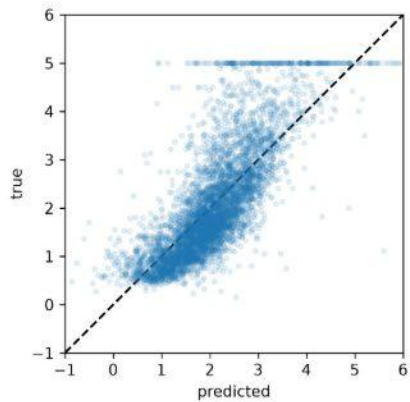


Prediction plots

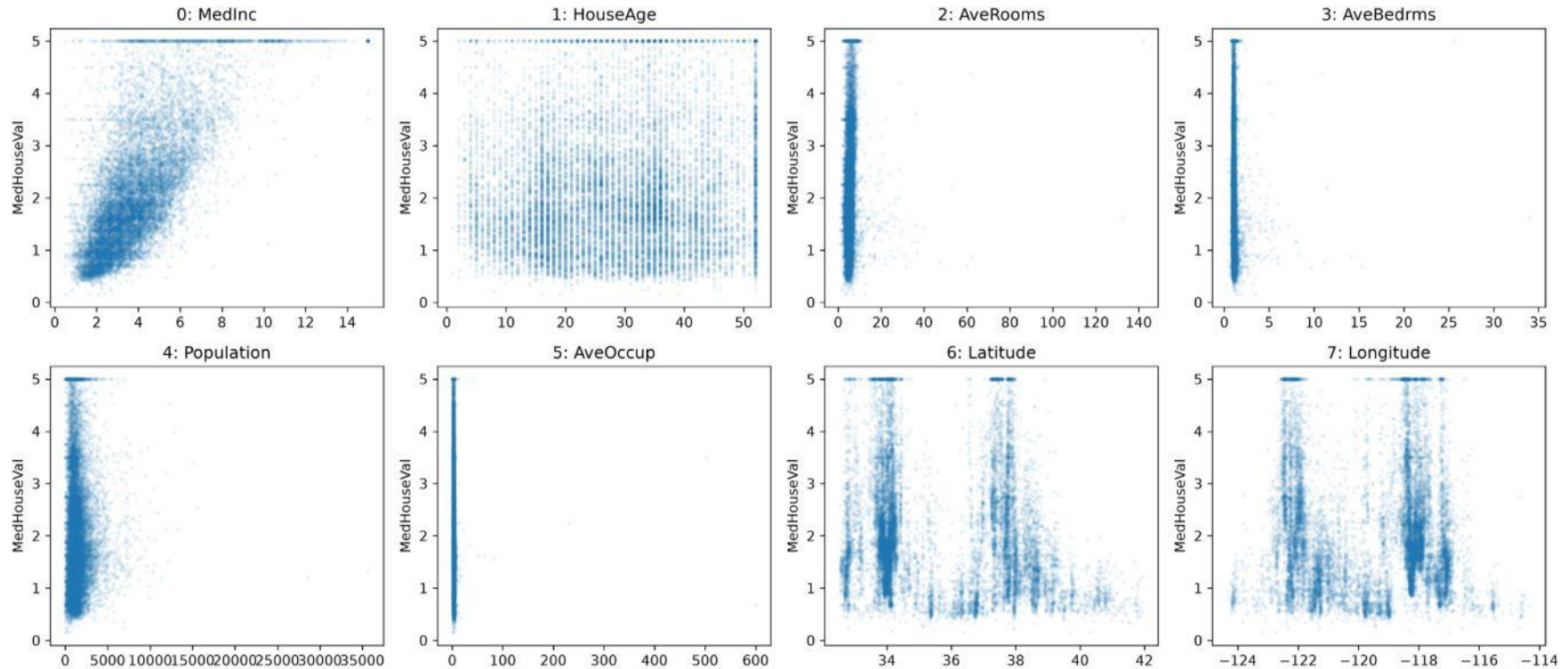
```
from sklearn.linear_model import Ridge
from sklearn.datasets import fetch_california_housing
housing = fetch_california_housing()
X_train, X_test, y_train, y_test = train_test_split(
    housing.data, housing.target, random_state=0)
ridge = make_pipeline(StandardScaler(), Ridge())
ridge.fit(X_train, y_train)
pred = ridge.predict(X_test)
plt.plot(pred, y_test, 'o', alpha=.1)
# or: PredictionErrorDisplay.from_predictions(
#     y_test, pred, kind="actual_vs_predicted")
```



Residual Plots



Target vs Feature



Residual vs Feature

